

第六届全国高校教师教学创新大赛：

AI 赋能 Java Web 应用开发课程

本参赛作品所有验收资料已汇总以下[网站](https://javaweb-ai-teaching.github.io/coursecompetition)，欢迎访问快速评审：

<https://javaweb-ai-teaching.github.io/coursecompetition>

目录

- 第六届全国高校教师教学创新大赛：1
- AI 赋能 Java Web 应用开发课程 1
- 一、 五维自研 AI 教学工具截图 3
 - 1.1 工具套件总览4
 - 1.2 智能助教 WebDev 助教7
 - 1.3 AI 自适应编程挑战 WebSec 挑战13
 - 1.4 团队协作教练 TeamCoach16
 - 1.5 人机对抗 Code Review Battle 21
 - 1.6 AI 辅助教学资料重构 31
- 二、 教学成效数据45
- 三、 学生作品与 AI 反馈样例 56

一、五维自研 AI 教学工具截图

AI赋能Java Web教学创新工具套件

Java Web应用开发 · 五维AI融合创新教学体系

AI-Empowered Java Web Teaching Innovation Toolkit



第六届全国高校教师教学创新大赛 · 人工智能赛道

专业核心课程 | 32学时 | 2学分 | 覆盖6个AI教学情境

1.1 工具套件总览





智能助教

课程专属AI智能体，24h个性化问答与学情分析，让课外学习不再无助。



AI自适应编程挑战

AI动态出题、自适应难度的编程挑战平台，让练习从被动到主动、从同质到个性。



团队协作教练

基于Git数据的AI团队分析，解决搭便车问题，实现过程性团队评价。



人机对抗

学生与AI同台审查代码，培养批判性思维，认识AI能力边界。



AI辅助教学资料重构

个性化实验设计生成器+AI课程PPT生成，从教师端释放备课压力。

六大教学情境覆盖矩阵：

赛道要求的教学情境	智能助教	编程挑战	团队教练	人机对抗	资料重构
① 学情数据采集与分析	★ 提问记录分析	★ 闯关数据追踪	★ Git提交分析		
② 数字资源整合与运用	★ RAG课程知识库	★ 动态题库生成			★ PPT与实验生成
③ 适配的教学场景设计		★ 自适应关卡	★ 个性化任务	★ 对抗场景设计	★ 一人一题
④ 多维智能评价反馈		★ 闯关评分	★ 贡献度报告	★ 人机评审对比	
⑤ 师生机协同教学	★ AI助教角色	★ AI出题+学生解		★ 人vs AI对战	
⑥ 个性化学习支持	★ 24h个性问答	★ 自适应难度	★ 薄弱点推荐		★ 分层实验任务



覆盖结论：五维AI融合创新覆盖全部6个教学情境，远超"至少2个"的赛道要求。每个情境均有2-3个维度交叉支撑，确保覆盖的深度与可信度。

1.2 智能助教 WebDev 助教

登陆界面:



WebDev助教

Java Web应用开发课程专属教学智能体 · 基于RAG + DeepSeek



学生入口

AI助教答疑 · 知识点查询 · 学习数据

学号

2023001

姓名

陈思远

进入学习



教师入口

学情分析 · 数据看板 · 权限管理

工号

T001

姓名

XXX老师

进入管理

Java Web应用开发课程 · 教学创新演示平台 · 2025-2026学年

选择学生入口，输入学号姓名点击进入学习后来到了界面：

WebDev助教 • Java Web应用开发

基于RAG + DeepSeek的课程知识库问答系统

模型在线 · 知识库已同步16讲

学生: 陈思远

退出登录

WebDev助教

Java Web应用开发 · 随时为你答疑解惑

20:39

如何实现用户登录状态的保持?

陈

20:45

W

这是 第5讲 会话管理 的实战应用场景，我来给你梳理完整的实现方案：

方案一：Session（服务端方案，最常用）

```
// 登录成功后，将用户信息存入Session HttpSession session = request.getSession(); session.setAttribute("currentUser", user); session.setMaxInactiveInterval(30 * 60); // 30分钟超时
// 判断是否已登录 Object user = session.getAttribute("currentUser"); if (user == null) { response.sendRedirect("/login.jsp"); }
```

方案二：Cookie + Session（记住我功能）

- 用户勾选“记住我”时，生成一个加密Token存入Cookie
- Cookie设置较长的过期时间（如7天）
- 下次访问时，Filter检查Cookie中的Token，自动恢复Session

方案三：登录拦截Filter（全局保护）

```
// LoginFilter - 统一拦截未登录访问 public void doFilter(...) { HttpServletRequest req = (HttpServletRequest) request; String uri = req.getRequestURI(); // 放行登录页和静态资源 if (uri.contains("/login") || uri.contains("/static")) { chain.doFilter(request, response); return; } // 检查Session中的登录状态 HttpSession session = req.getSession(false); if (session != null && session.getAttribute("currentUser") != null) { chain.doFilter(request, response); } else { ((HttpServletResponse)response).sendRedirect("/login.jsp"); }
```

这三个方案在 第15讲 项目实战(上) 中有完整代码实现，建议对照实验动手练习！

20:46

输入你的问题... 试问：Servlet生命周期是什么?

AI使用规范：AI是学习助手，不是答案机。请先独立思考，再向AI提问。

DeepSeek API · ChromaDB · RAG Pipeline · 课程知识库 (16讲) · Spring Boot · Vue 3 前端

我的学习数据

12 本周提问次数

78% 知识点掌握率

薄弱知识点

Session安全 · JDBC事务 · Filter链机制

学习建议 (AI生成)：建议复习第5讲会话管理和第14讲事务控制，重点关注Session安全管理的防劫持策略和JDBC事务的ACID特性。第9讲Filter链的执行顺序也需要加强理解。

我的学习轨迹（本周）

周一						3次
周二						2次
周三						5次
周四						2次
周五						0次
周六						4次
周日						3次

推荐练习 (AI推荐)

实现登录拦截Filter

第9讲 - 综合运用Filter和Session

中等

Session/Cookie对比练习

第5讲 - 10道选择题巩固概念

基础

JDBC事务回滚演示

第14讲 - 模拟银行转账事务

较难

PreparedStatement防注入

第8讲 - 对比Statement和PreparedStatement

中等

选择教师入口点击进入管理，输入工号姓名即可进入教师端：

WebDev助教 • Java Web应用开发

基于RAG + DeepSeek的课程知识库问答系统

模型在线 · 知识库已同步16讲

教师: XXX老师

切换到学生视图

退出登录

WebDev助教

Java Web应用开发 · 随时为你答疑解惑

W

你好！我是 **WebDev助教**，Java Web应用开发课程的AI教学助手。我熟悉课程全部16讲的内容，可以帮你解答课程相关问题、复习知识点、分析代码示例。有什么想问的吗？

20:15

陈

老师好，我想问一下Session和Cookie到底有什么区别？什么时候该用哪个？

20:17

W

这是个很好的问题！在 **第5讲 会话管理** 中我们详细讲解了这个知识点，我帮你梳理一下核心区别：

1. 存储位置不同

- Cookie** 存储在客户端浏览器中
- Session** 存储在服务器端（默认在内存中）

2. 安全性差异

- Cookie 可被用户查看和篡改，安全性较低
- Session 数据在服务器端，相对安全

3. 典型使用场景

- Cookie: 记住用户名、语言偏好等非敏感信息
- Session: 登录状态、购物车、验证码等敏感数据

// Session存储登录信息 HttpSession session = request.getSession(); session.setAttribute("currentUser", user);

记忆口诀：敏感数据用Session，偏好记忆用Cookie。

20:17

输入你的问题... 试试问：Servlet生命周期是什么？

会话管理

今日学情概览

今日活跃 42 / 58 人

72%

提问总数 127 条

平均对话轮次 3.8 轮

问题解决率 89.2%

AI使用统计

127 本周总提问数

18.1 日均提问

21-23 高峰时段

58 使用学生数

最活跃学生 TOP3

1 赵天宇 23次提问

2 陈思远 19次提问

3 林雨桐 17次提问

学生提问时间分布 (24小时)

本周知识点命中率

会话管理 92%

JDBC/数据库 85%

Filter/Listener 78%

MVC架构 75%

Web安全 71%

JSP/EL/JSTL 68%

Spring MVC 65%

MyBatis 58%

DeepSeek API

ChromaDB

RAG Pipeline

课程知识库 (16讲)

Spring Boot

Vue 3 前端

右侧可展示学情概览，包括：知识点热力图、提问频次、高频问题 TOP 等，还可以进行权限设置。

AI回答质量评估

4.2/5

自动评分平均

8.3%

需人工介入

91.7%

自动解决率

质量分布: 优秀(4.5+) 42.3% | 良好(3.5-4.5) 43.7% | 需改进(<3.5) 14.0%

待人工审核: 涉及课外延伸问题6条, 超纲回答3条, 表述不够清晰2条

预警学生 (超过5天未使用)

邓宇航 (2023042)

最后使用: 3月14日 | 累计提问: 3次 | 掌握率: 45%

发送提醒

罗晨曦 (2023028)

最后使用: 3月13日 | 累计提问: 5次 | 掌握率: 52%

发送提醒

宋雨泽 (2023051)

最后使用: 3月15日 | 累计提问: 2次 | 掌握率: 38%

发送提醒

学情热力图 • 知识点查询频次

	L1	L2	L3	L4	L5	L6	L7	L8	L9	L10	L11	L12	L13	L14	L15	L16
Servlet基础	低	低	低	低	低	低	低	低	低	低	低	低	低	低	低	低
JSP/EL/JSTL	低	低	中	高	低	低	低	低	低	低	低	低	低	低	低	低
会话管理	低	低	低	低	高	中	低	低	低	低	低	低	低	低	低	低
数据库/JDBC	低	低	低	低	低	低	中	高	低	低	低	低	低	低	低	低
框架/MVC	低	低	低	低	低	低	低	低	高	高	中	低	低	低	低	低
安全/部署	低	低	低	低	低	低	低	低	低	低	低	低	中	高	中	低

低 中 高

高频问题 TOP 5

1

Session和Cookie的区别是什么?

47次

2

MVC模式中Controller的职责?

38次

3

JDBC连接池原理及配置方法?

35次

4

Filter和Listener的执行顺序?

31次

5

如何防止SQL注入攻击?

28次

知识薄弱点预警

Session安全管理 正确率 34%

Filter链机制 正确率 41%

JSTL自定义标签 正确率 38%

JDBC事务控制 正确率 45%

MVC分层解耦 正确率 42%

XSS/CSRF防护 正确率 31%

学生提问记录 (近期)

时间	学生	问题摘要	涉及知识点	AI回答质量
21:47	赵天宇	Servlet的init()方法只调用一次吗?	Servlet	4.8
21:42	陈思远	PreparedStatement和Statement性能差异?	JDBC	4.6
21:38	林雨桐	Filter链的执行顺序由什么决定?	Filter	4.7
21:32	王浩然	JSP中EL表达式如何访问Map?	EL/JSTL	3.9
21:28	刘诗琪	Spring MVC的@RequestMapping用法?	Spring MVC	4.5
21:15	周雅婷	JDBC事务回滚的条件是什么?	JDBC事务	4.0
21:08	孙嘉禾	Cookie的Secure和HttpOnly属性区别?	会话管理	4.8
20:55	吴若晨	MyBatis的#{}和\${}有什么区别?	MyBatis	4.9
20:45	郑思齐	XSS攻击的防御方法有哪些?	Web安全	3.8

 权限设置

允许学生查看AI完整代码 ☐

允许课后使用 ☒

限制每日提问次数 (上限: 20次) ☒

启用敏感内容过滤 ☒

 知识库管理

- | | | | | | | | |
|-----|------|------|------|------|------|------|------|
| 第1讲 | 第2讲 | 第3讲 | 第4讲 | 第5讲 | 第6讲 | 第7讲 | 第8讲 |
| 第9讲 | 第10讲 | 第11讲 | 第12讲 | 第13讲 | 第14讲 | 第15讲 | 第16讲 |

 导出学情报告

1.3 AI 自适应编程挑战 WebSec 挑战

闯关主界面：

WS

WebSec挑战 · AI驱动的安全攻防闯关平台

每人题目不同 · AI动态生成 · 自动判定

48人在线

AI引擎 v2.3

陈思远
软件工程 2023级 | 2023010115

LESSON 14
数据库事务与连接池
1/5 已完成

LESSON 15
用户认证与权限控制
1/5 已完成

漏洞类型

SQL注入 (SQL Injection)

OWASP分类

OWASP Top 10 - A03:2021

难度

★★★★☆

知识点

StatementPreparedStatementDAO

学习目标

理解SQL注入的原理和危害，掌握使用PreparedStatement替代字符串拼接SQL的安全编码方法。

参考资料

LEVEL 1

难度: 简单

第1关: SQL注入基础

以下DAO代码使用字符串拼接构建SQL查询，存在严重的SQL注入漏洞。攻击者可以通过构造恶意输入绕过身份验证或获取敏感数据。请找到漏洞并编写修复代码。

UserDAO.java

设置

复制

1234567891011121314

```
public class UserDAO {
    private Connection connection;

    public User findByUsername(String username) {
        // BUG: 字符串拼接SQL，存在SQL注入漏洞
        String sql = "SELECT * FROM users WHERE username = '" + username + "'";
        Statement stmt = connection.createStatement();
        ResultSet rs = stmt.executeQuery(sql);
        if (rs.next()) {
            return mapToUser(rs);
        }
    }
}
```

闯关进度 - 第十四讲

第1关: SQL注入基础
✓ 已通过
3:15

第2关: 事务一致性
2 进行中
07:58

第3关: 连接池泄漏
即将开放
--:--

第4关: 批量操作优化
即将开放
--:--

第5关: 综合挑战 (BOSS关)
即将开放
--:--

本关用时

07:58

总用时 11:13

尝试次数 0

实时排行榜

48人参与

1 王浩然
第2关 06:51

2 林雨桐
第2关 07:14

3 刘诗琪
第2关 07:59

班级闯关进度 (实时) - 第十四讲

第1关 85%

第2关 35%

第3关 0%

第4关 0%

第5关 0%

漏洞类型掌握热力图

SQL注入 越权访问 Session XSS 综合

A班 85% 35%

B班 80% 30%

C班 72% 22%

课程整体安全知识掌握度

SQL注入 72%

越权访问

Session安全

XSS防护

事务管理 35%

AI教学预警

第2关 (事务一致性) 通过率较低 (35%)，建议补充讲解ACID事务特性和rollback用法。

发送课堂提醒

常见错误 TOP 3
#1 忘记调用conn.setAutoCommit(false) 55%
#2 catch块中未调用rollback() 35%

代码编辑器与漏洞修复：展示学生正在修复 AI 生成的漏洞代码、JUnit 测试反馈

AdminUserServlet.javaUser.java.web.xml

12345678910111213141516

```
@WebServlet("/admin/users")
public class AdminUserServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // ⚠️ BUG: 未检查用户角色，任何已登录用户都能访问管理页面
        HttpSession session = request.getSession();
        String username = (String) session.getAttribute("username");
        if (username != null) {
            List<User> users = userDAO.findAll();
            request.setAttribute("users", users);
            request.getRequestDispatcher("/WEB-INF/admin/users.jsp").forward(request, response);
        } else {
            response.sendRedirect("/login.jsp");
        }
    }
}
```

设置复制

修复方案

// 在此编写修复代码...

// 提示：需要在验证用户登录后，额外检查用户角色

// 例如：从session中获取role属性，判断是否为admin

提交修复

AI提示

重置

教师实时数据大屏：展示各关通过率、平均用时、排行榜、薄弱关卡预警以及 AI 教学预警（会汇总常见错误和后续建议）

班级闯关进度 (实时) - 第十五讲

第1关	92%	92%
第2关	40%	40%
第3关	12%	12%
第4关	0%	0%
第5关	0%	0%

漏洞类型掌握度热力图

	SQL注入	越权访问	Session	XSS	综合
A班	92%	48%	12%	--	--
B班	88%	35%	8%	--	--
C班	78%	28%	5%	--	--

课程整体安全知识掌握度

SQL注入	85%
越权访问	40%
Session安全	10%
XSS防护	5%
事务管理	--

AI教学预警

第2关通过率过低 (40%)，建议暂停讲解越权访问漏洞的核心概念。

发送课堂提醒

常见错误 TOP 3

#1 仅检查username非空，未检查role	62%
#2 使用==比较字符串而非equals()	28%
#3 返回401而非403状态码	15%

闯关进度

挑战记录

排行榜

实时排行榜

48人参与

1林林雨桐

第3关 07:21

2王王浩然

第3关 07:59

3赵赵天宇

第2关 08:40

4刘刘诗琪

第2关 08:56

5陈陈思远 (我)

第2关 10:23

6周周雅婷

第2关 09:49

7孙孙嘉禾

第2关 10:18

8吴吴若晨

第1关 10:44

9张张博文

第1关 11:09

闯关进度

挑战记录

排行榜

挑战记录

时间	关卡	结果	用时	尝试
14:32	L15-1 SQL注入	通过	2:34	2次
14:28	L15-1 SQL注入	失败	1:45	1次
13:55	L14-1 SQL注入基础	通过	3:15	1次
13:48	L14-1 SQL注入基础	失败	2:10	1次

我的安全技能雷达图

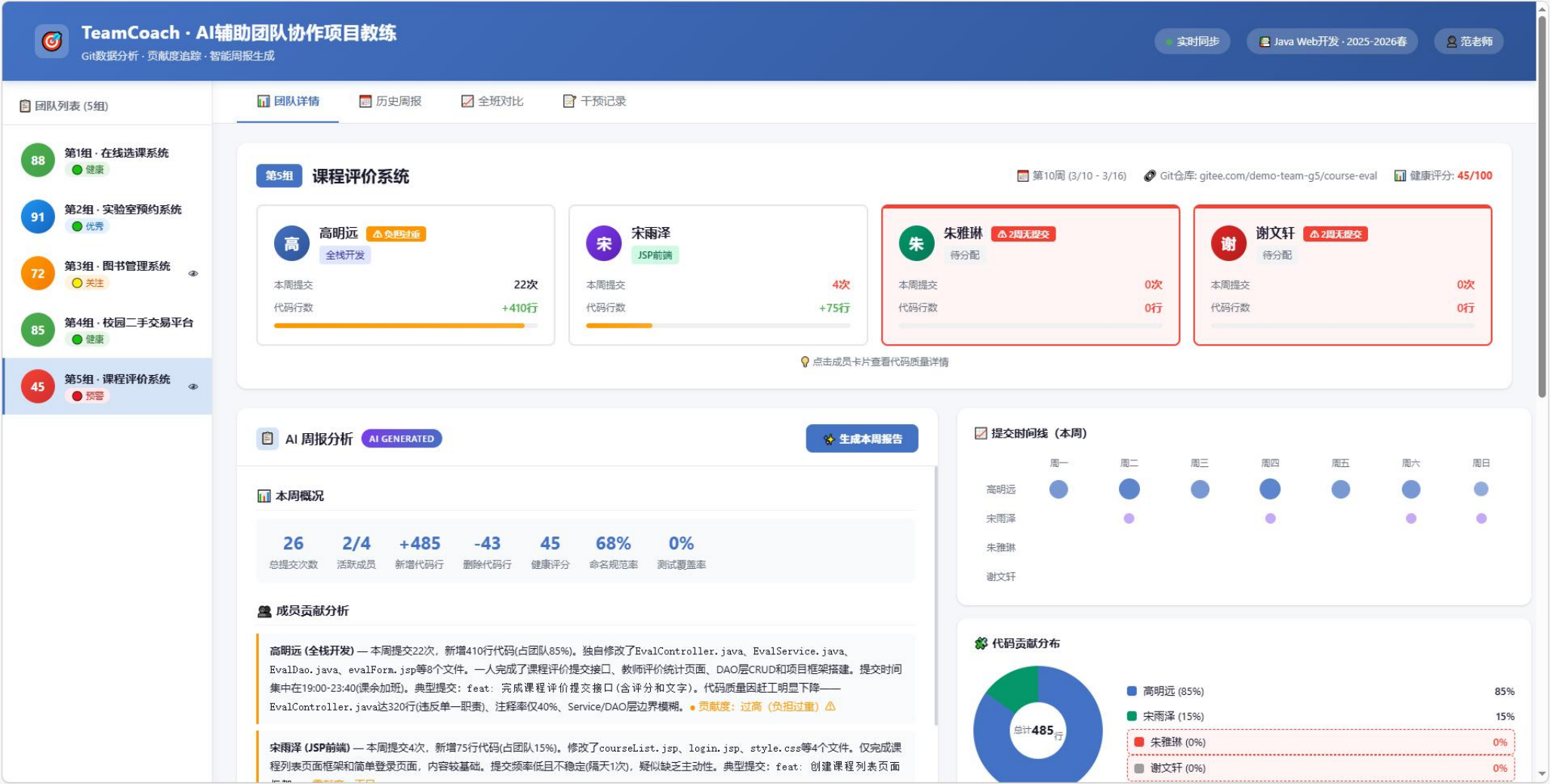
技能	等级
SQL注入	5
越权访问	4
Session	3
XSS	2
事务管理	3

全班排名趋势

排名	趋势
#10	开始
#9	上升
#8	上升
#7	上升
#6	上升
#5	结束

1.4 团队协作教练 TeamCoach

团队总览面板：展示各团队健康分、成员贡献度雷达图、搭便车预警



点击高明远同学的成员卡片即可查看代码质量详情:



AI 生成的团队周报：展示贡献度分析、分工合理性评估、风险提示、改进建议

本周概况

26 2/4 +485 -43 45 68% 0%

总提交次数 活跃成员 新增代码行 删除代码行 健康评分 命名规范率 测试覆盖率

成员贡献分析

高明远 (全栈开发) — 本周提交22次, 新增410行代码(占团队85%)。独自修改了EvalController.java、EvalService.java、EvalDao.java、evalForm.jsp等8个文件。一人完成了课程评价提交接口、教师评价统计页面、DAO层CRUD和项目框架搭建。提交时间集中在19:00-23:40(课余加班)。典型提交: feat: 完成课程评价提交接口(含评分和文字)。代码质量因赶工明显下降——EvalController.java达320行(违反单一职责)、注释率仅40%、Service/DAO层边界模糊。●贡献度: 过高 (负担过重) ⚠

宋雨泽 (JSP前端) — 本周提交4次, 新增75行代码(占团队15%)。修改了courseList.jsp、login.jsp、style.css等4个文件。仅完成课程列表页面框架和简单登录页面, 内容较基础。提交频率低且不稳定(隔天1次), 疑似缺乏主动性。典型提交: feat: 创建课程列表页面框架。●贡献度: 不足

朱雅琳 (待分配) — 连续2周0次提交, 0行代码。Giti记录显示最后活跃时间为3月5日(第8周), 内容为添加.gitignore文件。此后完全无任何活动记录。●贡献度: 零 ⚠

谢文轩 (待分配) — 连续2周0次提交, 0行代码。Giti记录显示最后活跃时间为3月6日(第8周), 内容为修改README.md文件。此后完全无任何活动记录。●贡献度: 零 ⚠

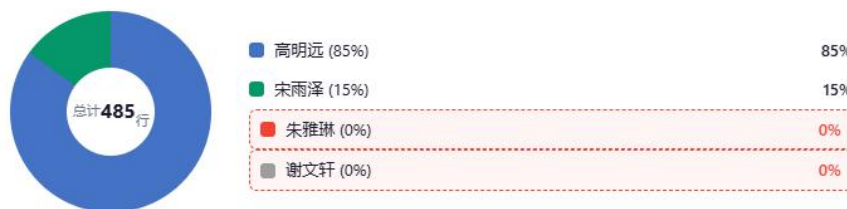
成员活跃时间线

各成员最后提交时间:

提交时间线 (本周)



代码贡献分布



代码质量趋势 (近4周)



🕒 成员活跃时间线

各成员最后提交时间:

高明远

最后提交: 03-16 23:40
本周每天活跃(19:00-23:40)

宋雨泽

最后提交: 03-16 20:30
本周隔天提交, 频率低

朱雅琳

最后提交: **03-05(12天前)**
△ 连续12天零活动

谢文轩

最后提交: **03-06(11天前)**
△ 连续11天零活动

🚨 严重风险警告

🔥 **团队濒临瘫痪** — 仅有1人 (高明远) 在进行实质性开发工作(85%代码量), 团队协作完全失效。若不立即干预, 项目有极高的失败风险。健康评分已连续3周下降(-17分)。

🔥 **朱雅琳、谢文轩完全失联** — 两位成员分别在3月5日和3月6日后无任何Git活动, 已超过11天。尚未分配任何开发任务, 疑似已放弃项目。建议教师立即联系确认情况, 同时检查课程群聊和邮件是否有回复。

⚠️ **高明远过度负担** — 独自完成85%的工作量, 提交时间集中在晚间19:00-23:40(明显在课余加班)。EvalController.java达320行, 注释率仅40%, 代码质量因赶工持续下降。

⚠️ **项目进度严重滞后** — 与其他团队相比, 核心功能完成度仅30%, 至少落后2周进度。测试覆盖率为0%。

📌 紧急建议 — 需要教师立即介入

🔴 **立即面谈朱雅琳和谢文轩** — 了解缺席原因(技术困难? 团队沟通障碍? 个人原因?), 明确下周必须完成的最低任务量。若无法参与, 需考虑调整分组或成绩评定方案。已安排周三下午面谈。

🤖 AI生成时间: 2026-03-16 08:00 · 数据来源: Gitee API

模型: DeepSeek-V3 · 置信度: 98%

📌 紧急建议 — 需要教师立即介入

🔴 **立即面谈朱雅琳和谢文轩** — 了解缺席原因(技术困难? 团队沟通障碍? 个人原因?), 明确下周必须完成的最低任务量。若无法参与, 需考虑调整分组或成绩评定方案。已安排周三下午面谈。

🔴 **重新分配任务** — 朱雅琳负责DAO层(数据库设计和CRUD), 谢文轩负责JSP前端页面开发, 每人每天至少提交2次。

🔴 **设立每日检查点** — 建议教师每天查看该团队的Git提交记录, 连续2天无提交立即约谈。

🟢 **帮助高明远减负** — 将已完成的evalForm.jsp和evalStats.jsp交由宋雨泽维护和优化, 高明远专注于核心业务逻辑和代码审查。拆分320行的EvalController为多个Controller。

🤖 AI生成时间: 2026-03-16 08:00 · 数据来源: Gitee API

模型: DeepSeek-V3 · 置信度: 98%

教师操作面板

发送提醒

高明远

发送提醒

宋雨泽 偏低

发送提醒

朱雅琳 无提交

发送提醒

谢文轩 无提交

发送提醒

团队管理

标记关注团队

安排面谈

团队群聊提醒

AI调整分工建议

AI SUGGESTION

该团队处于高风险状态，需要教师立即干预。核心问题：(1) 朱雅琳和谢文轩已分别11-12天未参与，必须面谈了解原因；(2) 高明远一人承担85%工作且提交集中在晚间加班，身心负担极大；(3) 项目进度落后全班至少2周。建议：立即安排面谈→重新分工→设立每日检查机制→帮助高明远减负。

1.5 人机对抗 Code Review Battle

对战规则与流程页：展示三轮对战流程（人工审查→AI 审查→对比辩论）

</>

Code Review Battle · AI代码评审对战

{ }

1 规则说明

2 代码审查

3 AI审查

4 对比分析

5 课堂辩论

6 总结反思

Code Review Battle 规则

第3轮对战 · 本轮主题：Servlet安全认证与过滤器 · 学生团队 vs AI

对战历史

第1轮
Servlet生命周期
学生 65 : AI 78

第2轮
JDBC连接管理
学生 80 : AI 90

第3轮 (本轮)
认证与过滤器安全
进行中...

学生团队累计

215

vs

AI累计

248

参赛分组

A组 认证组
陈思远、林雨桐、王浩然(组长)

B组 授权组
赵天宇、周雅婷、孙嘉禾(组长)

C组 视图一致性组
马文博、黄佳怡、杨子涵(组长)

对战目标

- 审查 LogoutServlet.java 和 RoleBasedAuthFilter.java 中的安全漏洞
- 比较人类审查员与AI审查的差异，理解各自优势与局限
- 掌握认证、授权、会话管理相关的安全编码最佳实践

- 掌握认证、授权、会话管理相关的安全编码最佳实践

评分标准

维度	满分	评分标准
发现数量	30分	每个有效发现10分，最高30分
严重性准确	30分	正确标注高危/中危/低危，每项10分
修复建议	20分	建议可行且具体，每项5-10分
知识点对应	20分	正确关联课程知识点（认证/授权/会话管理/HTTP安全），每项5分

流程



AI使用规则

本环节中AI作为对手而非助手，学生不得在审查阶段使用AI工具

开始对战

上一步

下一步

项目结构

src/

servlet/

LoginServlet.java

LogoutServlet.java

AdminServlet.java

filter/

RoleBasedAuthFilter.java

EncodingFilter.java

jsp/

login.jsp

index.jsp

学生审查倒计时 · 2个文件待审查

4:36

提交审查结果 →

LogoutServlet.java 文件1

RoleBasedAuthFilter.java 文件2

待审查代码 · LogoutServlet.java

安全审查任务

```
1 @WebServlet("/logout")
2 public class LogoutServlet extends HttpServlet {
3     protected void doGet(HttpServletRequest request, HttpServletResponse response)
4         throws ServletException, IOException {
5         HttpSession session = request.getSession(false);
6         if (session != null) {
7             // 问题1: 只移除了username, 未清除其他敏感属性(role, permissions)
8             session.removeAttribute("username");
9             // 问题2: 应使用session.invalidate()彻底销毁会话
10            // 问题3: 未设置Cache-Control头, 浏览器缓存可能导致未登录页面
11        }
12        response.sendRedirect("/login.jsp");
13        // 问题4: 重定向前未检查response是否已committed
14    }
15 }
```

提示: 点击代码行可添加审查标记

审查发现记录

全部文件

LogoutServlet.java

RoleBasedAuthFilter.java

行号

Logou

问题描述

严重程度

修复建议

×

+ 添加发现

← 上一步

下一步 →

AI 审查进行中

审查完成！共发现 8 个问题

Prompt 输入

请审查以下Java Web代码的安全漏洞，包括但不限于：认证绕过、授权缺陷、会话管理问题、输入验证不足、HTTP安全头缺失等。对每个发现标注严重性等级（高危/中危/低危），并给出具体的修复建议和对应的安全知识点。

// 附加文件: LogoutServlet.java, RoleBasedAuthFilter.java

正在分析代码结构...

检测到 LogoutServlet.java 中 2 个潜在问题

检测到 RoleBasedAuthFilter.java 中 3 个潜在问题

正在评估风险等级...

生成审查报告，共发现 8 个问题

LogoutServlet.java

1 高危

LogoutServlet.java

Line 7-8

Session未完全销毁 — 只移除了username，未清除其他敏感属性(role, permissions, isLoggedIn)，存在会话残留风险。攻击者可能利用残留的会话属性绕过权限检查。应使用session.invalidate()彻底销毁会话。

知识点: 会话管理 - Session生命周期与安全销毁

2 高危

LogoutServlet.java

Line 9

缺少Cache-Control/Pragma响应头 — 浏览器后退可能显示已登录页面，导致敏感信息泄露。应设置 response.setHeader("Cache-Control", "no-cache, no-store, must-revalidate") 等防缓存头。

知识点: HTTP安全 - 响应头安全配置

3 中危

LogoutServlet.java

Line 4

空Session处理逻辑 — getSession(false)返回null时仍执行重定向，逻辑可简化。建议增加null检查，区分已登录用户注销和未登录用户访问注销接口两种场景。

学生团队审查结果

Human Review · 3组9人 · 5 findings

LOGOUTSERVLET.JAVA

- 高危** **共识** LogoutServlet Line 7
只移除username未清除全部Session属性 — A组 陈思远 发现
- 中危** **共识** LogoutServlet Line 8
使用removeAttribute而非invalidate — B组 孙晓禾 发现
- 建议** **人类** LogoutServlet Line 11
重定向路径建议使用常量定义 — C组 杨子涵 发现

ROLEBASEDAUTHFILTER.JAVA

- 高危** **共识** AuthFilter Line 7-8
只检查登录状态未检查角色, 存在越权风险 — B组 赵天宇 发现
- 中危** **人类** AuthFilter Line 10
重定向路径硬编码, 未使用contextPath — A组 王浩然 发现

发现 **5** 个问题, 命中 **3** 个关键漏洞

VS

AI审查结果

AI Review · 2 files · 8 findings

LOGOUTSERVLET.JAVA

- 高危** **共识** LogoutServlet Line 7-8
Session未完全销毁, 存在会话残留风险
- 高危** **学习** LogoutServlet Line 9
缺少Cache-Control/Pragma响应头
- 中危** **学习** LogoutServlet Line 4
getSession(false)返回null时仍执行重定向, 逻辑可简化
- 建议** **学习** LogoutServlet 新增
建议添加CSRF token验证防止跨站注销

ROLEBASEDAUTHFILTER.JAVA

- 高危** **共识** AuthFilter Line 7-8
权限提升漏洞 — 只检查登录状态未检查角色
- 中危** **学习** AuthFilter Line 4
未安全转换请求类型, 缺少instanceof检查
- 中危** **共识** AuthFilter Line 10
重定向路径硬编码, 未使用context path
- 误报** **AI误报** LogoutServlet REJECTED
建议使用POST方法替代GET进行注销操作

发现 **8** 个问题, 命中 **4** 个关键漏洞, **1** 个误报

对比分析阶段还会进行发现重合度分析

发现重合度分析



3项

人机共识

1项

仅学生发现

3项

仅AI发现

1项

AI误报

A组 (认证组)
命中 **2/3** 关键漏洞

B组 (授权组)
命中 **3/3** 关键漏洞

C组 (视图一致性组)
命中 **1/2** 关键漏洞

← 上一步

下一步 →



规则说明



代码审查



AI审查



对比分析



课堂辩论



总结反思

课堂辩论引导

请各组讨论以下问题，每题2分钟，然后分享观点

4:12

问题 1 — HTTP安全头

AI发现的Cache-Control问题，为什么我们没有注意到？这说明什么？

▶ 展开参考答案

问题 2 — AI误报分析

AI建议用POST替代GET做注销，你同意吗？为什么？

▶ 展开参考答案

问题 3 — 授权漏洞

RoleBasedAuthFilter中的角色检查缺失，在实际系统中可能造成什么具体危害？如何设计一个完善的RBAC过滤器？

▶ 展开参考答案

问题 4 — FILTER安全漏洞

AI指出Filter中ServletRequest强转缺少instanceof检查，这是否属于过度防御？在什么场景下这个问题会真正产生影响？

▶ 展开参考答案

问题 5 — 人机协作策略

如果你是项目负责人，AI的8个建议中你会采纳哪几个？如何建立高效的人机协作代码审查流程？

▶ 展开参考答案

🏆 本轮精彩发言

周雅婷同学（8组）指出：AI遗漏了Filter中的类型转换风险在特定容器环境下可能被放大，而且在单元测试场景中如果使用不当的Mock对象也会触发ClassCastException。这个观点展示了对代码质量与安全的深层理解，获得本轮“最佳洞察”认定。

📝 学生观点记录

教师可在此记录学生讨论中的关键观点和精彩发言...

示例：

- 王浩然提出可以结合静态分析工具自动化检查硬编码路径

← 上一步

下一步 →

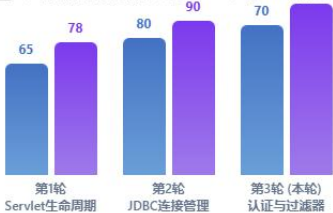
🦾 对战结果总览

维度	学生	AI	胜出
发现数量	5	8	🐱 AI
关键漏洞命中	3 / 4	4 / 4	🐱 AI
误报数	0	1	👤 学生
业务理解	★★★★	★★	👤 学生
跨文件关联	★★	★★★★	🐱 AI

🧡 结论：人机协同 > 单独任何一方

AI擅长系统性扫描，人类擅长业务判断 — 最佳策略是协作而非替代

📊 本学期对战数据总览 — 本轮 vs 前两轮



■ 学生团队 ■ AI

📈 趋势分析：学生团队得分从第1轮65分提升至本轮70分（+7.7%），关键漏洞命中率从50%提升至75%，呈现稳步提升趋势

✓ 规则说明

✓ 代码审查

✓ AI审查

✓ 对比分析

✓ 课堂辩论

6 总结反思

学生个人参与度评分

姓名	分组	发现数	关键命中	发言参与	本轮得分	累计得分
王浩然 组长	A组	2	1	积极	28	78
陈思远	A组	1	1	一般	24	70
林雨桐	A组	1	0	一般	18	62
孙嘉禾 组长	B组	2	2	积极	32	88
赵天宇	B组	2	1	积极	26	76
周雅婷	B组	1	1	突出	30	82
杨子涵 组长	C组	1	0	积极	20	66
马文博	C组	1	1	一般	22	68
黄佳怡	C组	0	0	一般	16	58



AI覆盖面广，善于发现模式化漏洞和跨层次安全问题



人类更擅长业务上下文判断和误报识别



最佳实践：AI初筛 + 人工复核 = 最优代码审查

● 本轮数据已记录，将纳入课程数据分析系统

导出本轮对战报告(PDF)

教学洞察 · Teaching Insights

🔍 AI找到 但学生没找到的

Cache-Control头问题 — 浏览器后退可查看已登录页面，属于安全盲区。学生普遍对Cache-Control头内容不敏感。

💡 学生找到 但AI遗漏的

学生对重定向路径的工程实践建议（常量、contextPath使用）更具体，体现了对跨层信任链的理解。AI虽然也发现了重定向问题。

⚠️ AI的误报分析

POST替代GET建议 — 注销操作使用GET是行业惯例（如Spring Security默认行为）。AI过度建议，这提醒学生，AI建议需要结合业务。

总结反思阶段最后会进行教学洞察：

教学洞察 · Teaching Insights

🔍 AI找到 但学生没找到的

Cache-Control头问题 — 浏览器后退可查看已登录页面，属于安全盲区。学生普遍缺乏HTTP头安全意识。

类型安全转换 — Filter中的强制类型转换未做防御性检查。

📖 学习机会

👨‍🎓 学生找到 但AI遗漏的

学生对重定向路径的工程实践建议（常量化、contextPath使用）更具体，体现了对项目维护性的理解。AI虽然也发现了硬编码问题，但学生的修复建议更贴近实际开发规范。

— 人类工程经验优势

⚠️ AI的误报分析

POST替代GET建议 — 注销操作使用GET是行业惯例（如Spring Security默认行为），AI过度建议。这提醒学生：AI建议需要结合实际工程经验做判断。

❌ 需要人工判断

← 上一步

1.6 AI 辅助教学资料重构

工具套件总览（含资料重构说明）：一人一题个性化实验设计生成器与 AI 课件生成的工作流程

 [方案概述](#)

 [两种技术路线](#)

 [PPT重构流程](#)

 [Beamer 方式](#)

 [Marp 方式](#)

 [自动化发布](#)

 [完整示例](#)

方案概述

教学PPT制作是教师最耗时的工作之一。传统使用 PowerPoint 制作PPT，效率低、版本管理混乱、难以复用。本方案使用 **纯文本**（LaTeX Beamer 或 Markdown）配合 **Cursor AI**，实现PPT的快速生成、智能重构和版本控制。

核心优势：

- **纯文本编辑**：使用 LaTeX/Markdown，可以用 Git 版本控制
- **Cursor 内完成**：编辑、预览、生成一站式完成，无需切换软件
- **AI 智能重构**：根据教学大纲和旧PPT内容，AI 自动优化结构
- **自动添加示例**：AI 生成合适的代码示例和图表
- **一键发布**：自动上传到 GitHub/Gitee，团队共享

适用场景



新课程PPT制作

从教学大纲快速生成完整PPT框架



旧PPT重构

优化老旧PPT的结构和内容



自动添加示例

AI生成代码示例和可视化图表



美化与排版

专业的学术风格模板



版本管理

Git管理，多人协作



一键发布

自动推送到代码仓库



两种技术路线对比

LaTeX Beamer

方式一：LaTeX Beamer

使用 LaTeX 的 Beamer 宏包制作学术风格的PPT

优势

- 数学公式排版完美（适合算法、理论课）
- 学术风格专业
- 代码高亮美观
- 输出PDF质量高

劣势

- 学习曲线较陡
- 编译速度较慢
- 调试复杂

适用课程

数据结构、算法设计、编译原理等理论性强的课程

Markdown (Marp)

方式二：Markdown + Marp

使用 Markdown 语法 + Marp 插件制作现代风格的PPT

优势

- 语法简单，易于上手
- 实时预览，所见即所得
- 编译速度快
- 样式现代、灵活

劣势

- 数学公式不如 Beamer 完美
- 自定义样式需要CSS知识

适用课程

Java Web、数据库、软件工程等实践性强的课程



推荐策略：根据课程特点选择合适的工具。对于数学公式多的理论课，选择 **Beamer**；对于代码示例多的实践课，选择 **Marp**。两种方式都可以在 Cursor 中完成，都支持 AI 辅助和实时预览。



PPT 重构的完整工作流程

以下展示如何使用 Cursor AI 将一份旧的 PowerPoint PPT（或现有的教学内容）重构为高质量的 Beamer/Marp PPT。

1

准备原始材料

收集旧PPT的内容（可以是 .pptx 导出的文本）、教学大纲、课程要点等材料。

PPT 文本提取

2

在 Cursor 中新建文件

创建 .tex (Beamer) 或 .md (Marp) 文件，选择合适的模板。

Cursor

3

AI 分析与重构

使用 Cursor AI (Ctrl+K) 分析原始内容，按照教学大纲重新组织结构。

Cursor AI

4

AI 生成示例代码

让 AI 为每个知识点生成合适的代码示例、图表、动画。

GPT-4

5

实时预览调整

使用 Cursor 的预览功能实时查看效果，边写边调整。

PDF 预览

6

编译生成PDF

在 Cursor 终端中编译生成最终的 PDF 文件。

XeLaTeX / Marp CLI

7

版本控制

提交到 Git 仓库，记录每次修改，方便回溯和协作。

Git

8

自动发布

推送到 GitHub/Gitee，自动发布到课程网站或分享给学生。

GitHub Actions

方式一：LaTeX Beamer 制作流程

在 Cursor 中配置 Beamer 环境



前置准备：

- 安装 TeX Live 或 MiKTeX (LaTeX 发行版)
- 在 Cursor 中安装 LaTeX Workshop 扩展
- 配置 PDF 预览功能

在 Cursor 中实时预览

1

安装 LaTeX Workshop

在 Cursor 扩展商店搜索 "LaTeX Workshop"，点击安装

2

配置编译命令

在 settings.json 中配置使用 XeLaTeX 编译 (支持中文)

3

Ctrl+S 保存即编译

保存文件后自动编译，几秒钟后在侧边栏看到 PDF 预览

4

实时调整

修改代码，保存后立即看到效果，快速迭代



时间对比：

- 传统 PowerPoint：制作一节课 (40-50页) PPT 需要 3-4 小时
- Beamer + Cursor AI：同样的内容只需 30-40 分钟
- 效率提升：约 5-6 倍

使用 Cursor AI 重构 Beamer PPT

Cursor 提示词：重构旧PPT

Cursor Prompt

请帮我将以下PPT内容重构为 LaTeX Beamer 格式：

【原始PPT内容】

[粘贴旧PPT的文字内容]

【教学大纲要点】

1. Servlet 生命周期
2. doGet 和 doPost 方法
3. 请求参数获取
4. 响应内容设置
5. 实际应用案例

【重构要求】

1. 按照教学大纲重新组织内容结构
2. 每个知识点一个 section
3. 为每个知识点添加：
 - 概念解释 (简洁清晰)
 - 代码示例 (完整可运行)
 - 应用场景说明
4. 使用 Madrid 主题
5. 添加适当的动画效果 (`\pause`, `\onslide` 等)
6. 确保中文显示正常

请生成完整的 .tex 文件代码。



方式二：Markdown + Marp 制作流程

🔧 在 Cursor 中配置 Marp



安装步骤：

- 在 Cursor 扩展商店搜索 "Marp for VS Code", 点击安装
- 安装 Marp CLI: `npm install -g @marp-team/marp-cli`
- 就这么简单! 无需复杂配置

👁 Marp 实时预览

1

打开预览窗口

点击右上角的 "Open Preview" 图标, 或按 `Ctrl+Shift+V`

2

分屏显示

左侧编辑 Markdown, 右侧实时预览 PPT 效果

3

边改边看

修改代码后自动刷新预览, 所见即所得

4

导出PDF

预览满意后, 右键选择 "Export Slide Deck"

🤖 使用 Cursor AI 生成 Marp PPT

Cursor 提示词: 生成 Marp PPT

Cursor Prompt

请根据以下教学大纲, 生成一份完整的 Marp 格式 PPT:

【课程信息】

- 课程: Java Web 开发
- 章节: 第5讲 - Session 和 Cookie
- 学时: 2学时 (90分钟)
- 授课对象: 大三学生

【教学大纲】

- 会话跟踪技术概述
 - 为什么需要会话跟踪?
 - HTTP 无状态协议的问题
- Cookie 技术
 - Cookie 的工作原理
 - Cookie 的创建和读取
 - Cookie 的属性设置
- Session 技术
 - Session 的工作原理
 - Session 的创建和使用
 - Session 与 Cookie 的关系
- 实际应用
 - 用户登录状态保持
 - 购物车实现
 - 防止重复提交

【生成要求】

- 使用 Marp 的 Markdown 语法
- 每个知识点包含:
 - 概念讲解 (简洁清晰)
 - 代码示例 (完整可运行的 Java 代码)
 - 图解说明 (用文字描述或 Mermaid 图)
- 添加课堂练习题 (1-2个)
- 总页数控制在 30-40 页
- 使用专业的样式 (可以自定义 CSS)

请生成完整的 Marp Markdown 文件。



自动化发布到 GitHub/Gitee

项目结构

推荐的项目目录结构

File Structure

```
teaching-slides/
├── java-web/                # Java Web 课程
│   ├── lecture-01-intro.md  # 第1讲 (Marp)
│   ├── lecture-02-servlet.md # 第2讲 (Marp)
│   ├── lecture-03-session.md # 第3讲 (Marp)
│   └── pdf/                 # 编译后的PDF
├── algorithm/              # 算法课程
│   ├── lecture-01-intro.tex  # 第1讲 (Beamer)
│   ├── lecture-02-sort.tex   # 第2讲 (Beamer)
│   └── pdf/
├── .github/workflows/       # GitHub Actions
│   └── build.yml            # 自动编译和发布
├── README.md
└── .gitignore
```

使用 Cursor 的 Git 功能

1

初始化 Git 仓库

在 Cursor 终端中: `git init`
或使用 Cursor 的源代码管理面板 (Ctrl+Shift+G)

2

编辑并预览PPT

在 Cursor 中编辑 .tex 或 .md 文件, 实时预览效果

3

编译生成PDF

Beamer: 自动编译
Marp: `marp lecture.md --pdf`

4

提交到 Git

在源代码管理面板中, 添加更改说明, 点击“提交”

5

推送到远程仓库

点击“同步更改”按钮, 自动推送到 GitHub/Gitee

6

自动发布 (可选)

配置 GitHub Actions, 自动编译和发布PDF到 Release

GitHub Actions 自动编译配置

.github/workflows/build.yml

YAML

```
name: Build Slides

on:
  push:
    branches: [ main ]

jobs:
  build-marp:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Marp Build
        uses: docker://marpteam/marp-cli:latest
        with:
          args: java-web/*.md --pdf --allow-local-files -o pdf/

      - name: Upload PDFs
        uses: actions/upload-artifact@v3
        with:
          name: slides-pdf
          path: pdf/*.pdf

      - name: Create Release
        uses: softprops/action-gh-release@v1
        if: startsWith(github.ref, 'refs/tags/')
        with:
          files: pdf/*.pdf
```



自动化效果:

- 在 Cursor 中编辑完PPT, 保存
- 使用 Cursor 的 Git 功能提交并推送
- GitHub Actions 自动编译所有 Markdown 为 PDF
- PDF 自动发布到 Releases, 学生可以直接下载
- 全程无需手动操作!

推送到 Gitee (国内访问更快)

在 Cursor 终端中配置 Gitee 远程仓库

Bash

```
# 添加 Gitee 作为第二个远程仓库
git remote add gitee https://gitee.com/your-username/teaching-slides

# 同时推送到 GitHub 和 Gitee
git push origin main
git push gitee main

# 或者配置一次推送到多个仓库
git remote set-url --add --push origin https://gitee.com/your-username/teaching-slides
git push # 一次推送到两个仓库
```



完整示例：在 Cursor 中使用 AI 重构 PPT

场景：重构 "Session 与 Cookie" 这一讲

Step 1: 准备原始内容（粘贴到 Cursor）

Text

=== 旧PPT内容 ===

第一页：会话跟踪

- HTTP是无状态的
- 需要会话跟踪技术

第二页：Cookie

- Cookie 是什么
- Cookie 的作用

第三页：Session

- Session 是什么
- Session 的使用

（内容过于简单，缺少代码示例和实际应用）

Step 2: 准备教学大纲要求

Text

=== 教学大纲 ===

知识点：

1. HTTP 无状态协议的问题
2. Cookie 的工作原理和 API
3. Session 的工作原理和 API
4. Cookie 与 Session 的区别
5. 实际应用：用户登录、购物车

要求：

- 每个知识点要有完整的代码示例
- 要有流程图说明
- 要有实际的应用案例（完整代码）

Step 3: Cursor AI 重构提示词

Cursor Prompt (Ctrl+K)

请将以下旧PPT内容重构为 Marp 格式的现代化教学PPT:

【旧PPT内容】

[粘贴上面的旧PPT内容]

【教学大纲要求】

[粘贴教学大纲]

【重构要求】

1. 按照教学大纲重新组织内容结构
2. 为每个知识点添加:
 - 清晰的概念解释 (用通俗易懂的语言)
 - 完整的 Java 代码示例 (可直接运行)
 - 流程图或架构图 (使用 Mermaid 语法)
 - 注意事项或常见错误
3. 添加实际应用案例:
 - 用户登录状态保持 (完整代码)
 - 购物车实现 (核心代码)
4. 添加课堂互动:
 - 每个 section 后添加思考题
 - 添加 1-2 个课堂练习
5. 使用 Marp 语法, 风格现代、专业
6. 总页数控制在 35-40 页

请生成完整的 Marp Markdown 文件。



重构效果:

- 从 3 页扩充到 20+ 页, 内容更加详实
- 添加了 5 个完整代码示例
- 添加了 2 个 Mermaid 流程图
- 添加了课堂练习和思考题
- 结构清晰, 逻辑连贯



参考示例与资源

示例文件下载



Beamer 示例

[查看 Beamer PPT 示例一](#)



Marp 示例

[查看 Marp PPT 示例一](#)

有用的资源链接

资源	说明
Marp 官方文档	https://marpit.marp.app/
Beamer 用户手册	https://ctan.org/pkg/beamer
Mermaid 图表语法	https://mermaid.js.org/
LaTeX 数学公式	https://katex.org/

最佳实践建议



保持源文件简洁

使用 Git 管理，避免一个 PPT 有多个版本文件



模块化组织

将常用的代码片段、图表保存为独立文件，方便复用



定期备份

推送到 GitHub/Gitee，既是备份又是版本控制



团队协作

多位教师可以通过 Git 分支协作，共同完善 PPT

展示内容

- 特性，概念，跟其他数据库对比，应用场景
- 使用教程：安装，使用，案例，常见数据库操作的实现方式

课程内容

- 构建数据库系统
- 数据库开发
- 数据库管理和维护
- 应用程序开发
- 其他数据库系统

Marp 示例:

Java Web 开发 |

Web的核心脉络—HTTP协议与Tomcat的使命

The Core of Web: HTTP Protocol & Tomcat's Mission

Java Web 开发 (Java Web Application Development)

HTTP协议 - Web的通用语 (HTTP: Universal Language of the Web)

1. HTTP 请求 (The Request)

客户端发送给服务器的"信件" (A "letter" from client to server)

- **请求行 (Request Line):** `GET /index.html HTTP/1.1`
 - 我想用 `GET` 方法, 获取 `/index.html` 资源
 - (I want to `GET` the `/index.html` resource)
- **请求头 (Headers):** `Host: www.tyust.edu.cn`
 - 附加信息, 如我从哪个主机来
 - (Additional info, like which host I'm from)
- **请求体 (Body):**
 - 发送的具体数据 (GET请求通常为空白)
 - (Specific data being sent - usually empty for GET requests)

二、教学成效数据

图3 WebSec安全攻防闯关平台通关率数据 (N=58)

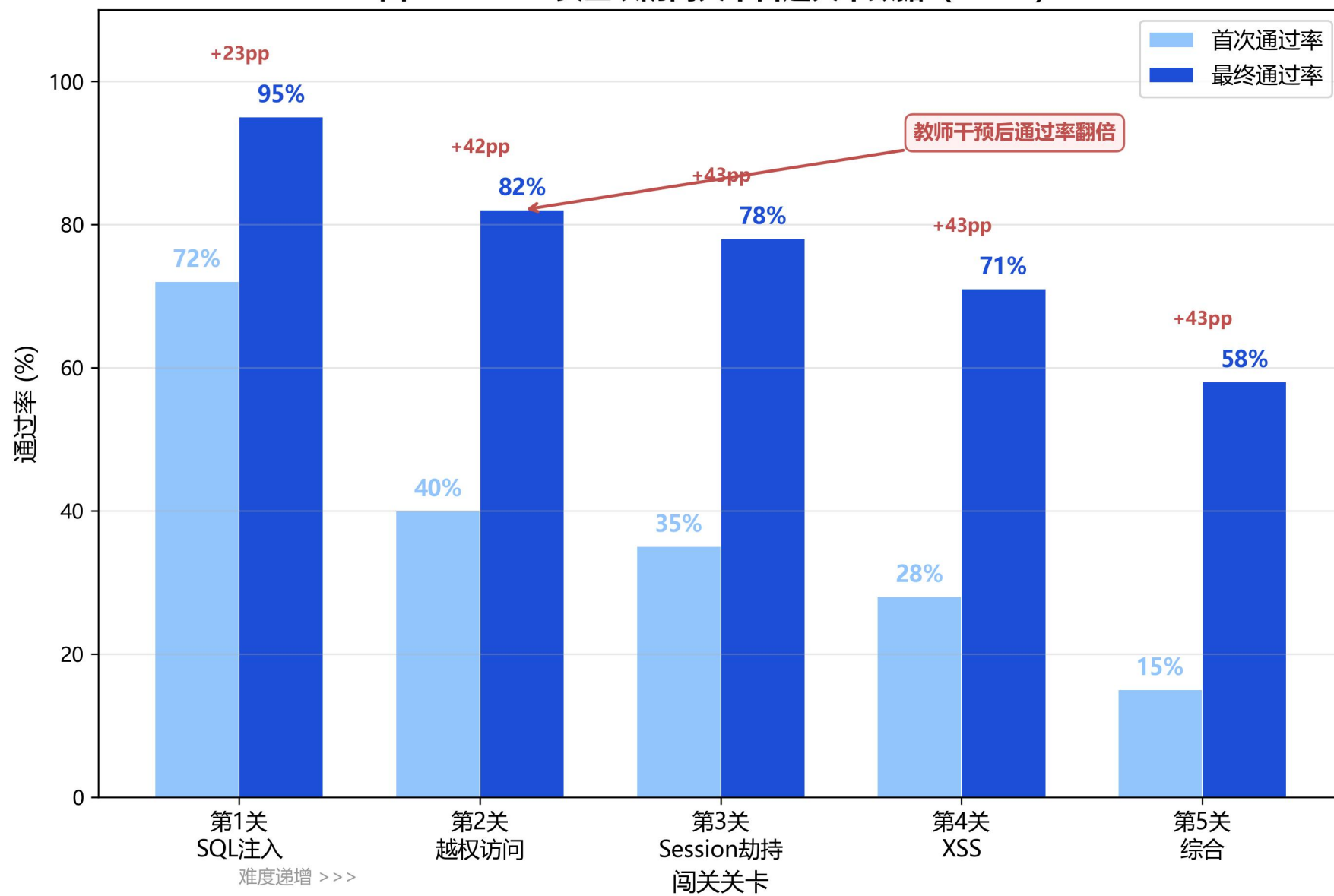


图4 TeamCoach团队贡献度均衡化趋势

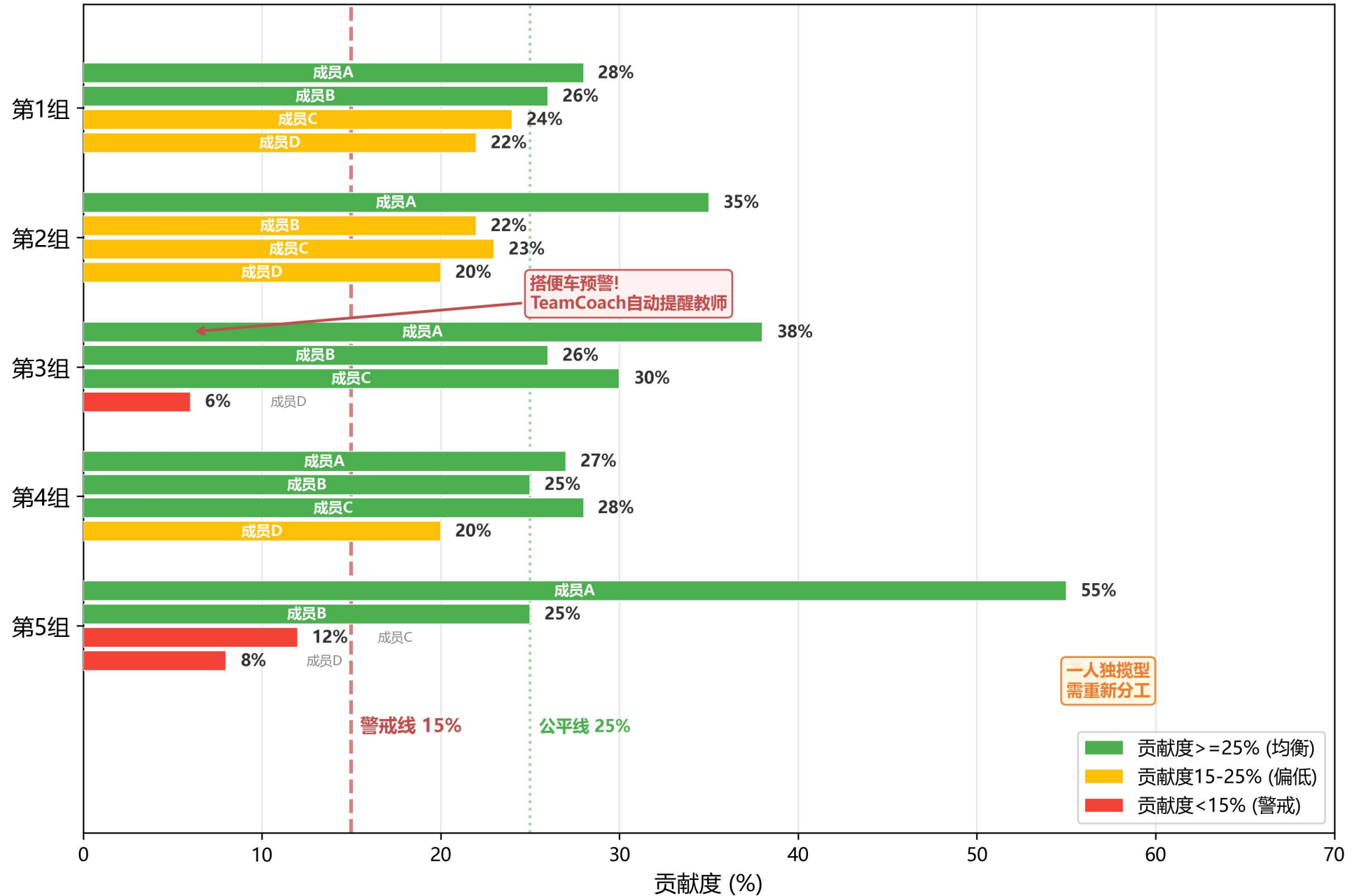


图5 Code Review Battle人机审查发现对比 (N=10组)

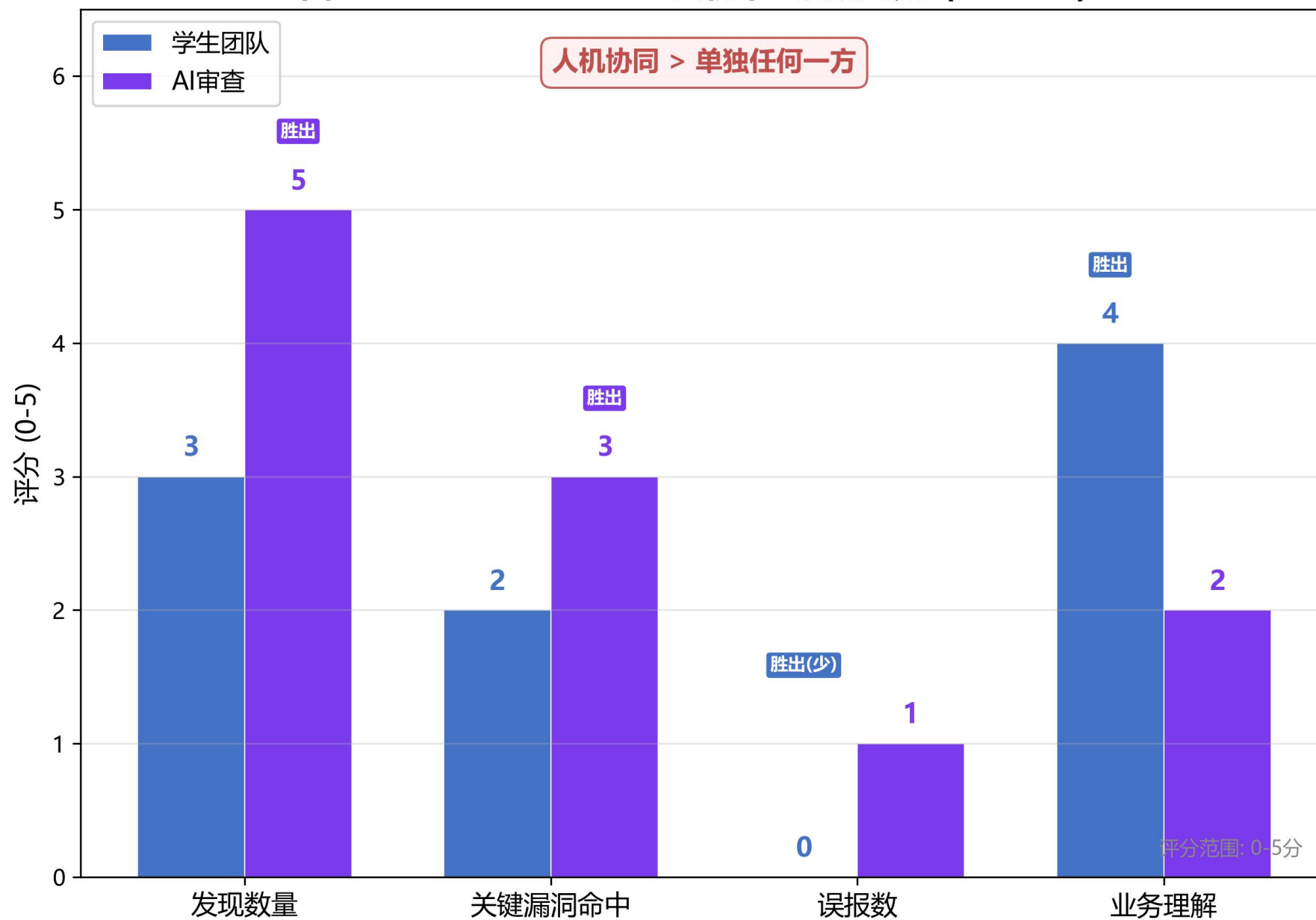


图6 MOSS代码相似度逐批次下降趋势 (2024 vs 2025)

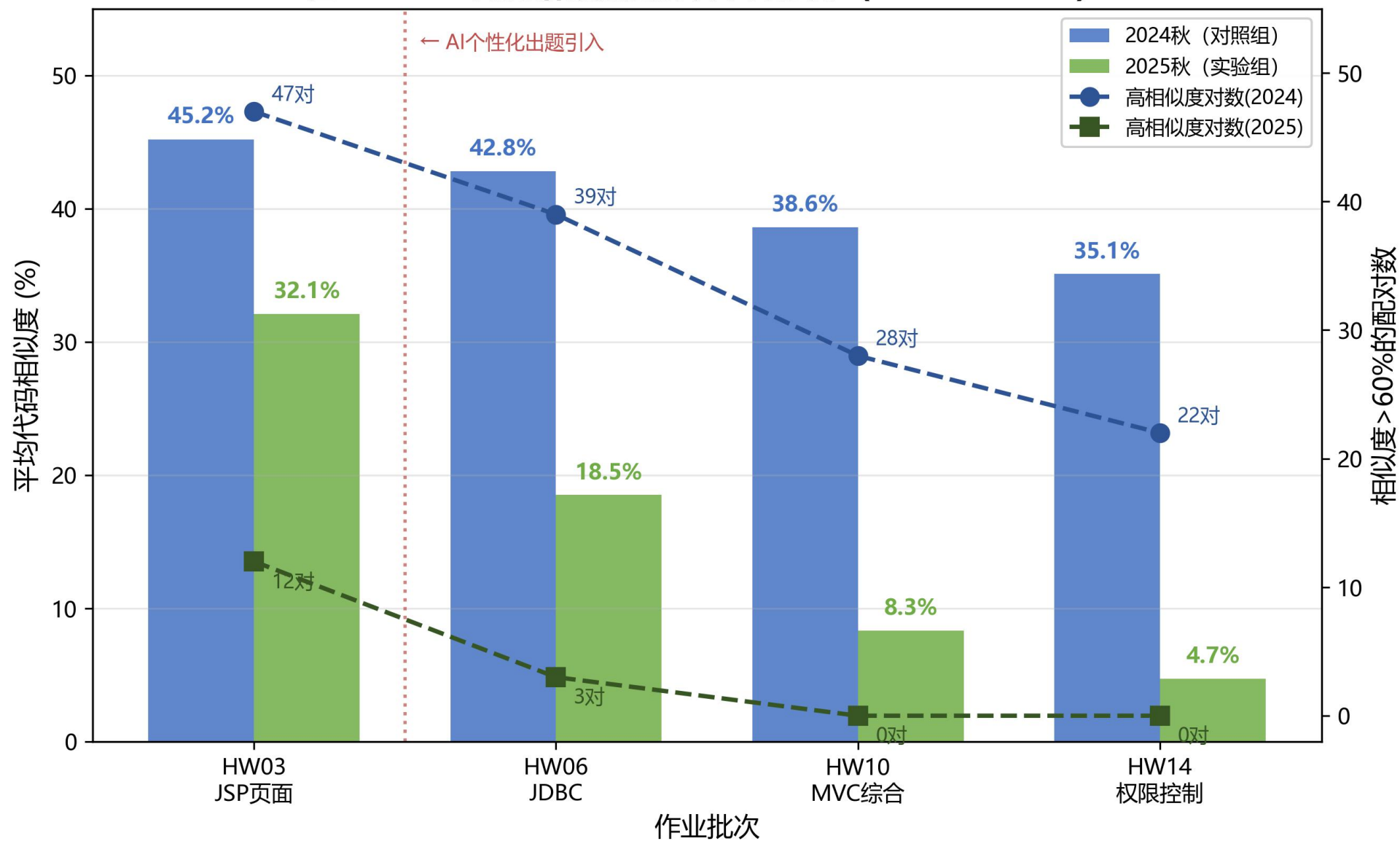


图7 AI辅助前后教师单份作业审查时间对比

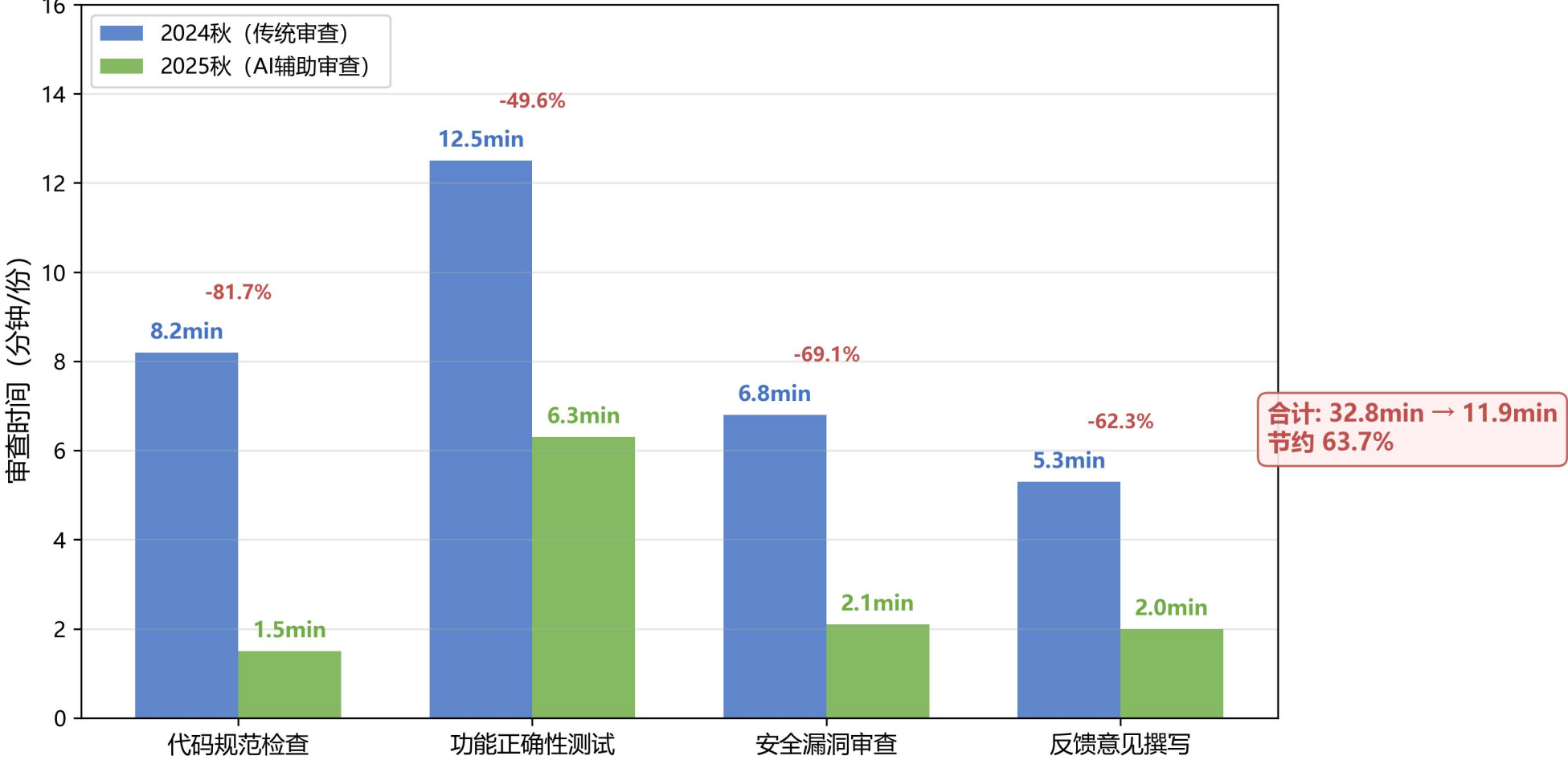


图8 SQL注入安全意识四时间点纵向追踪 (N=58)

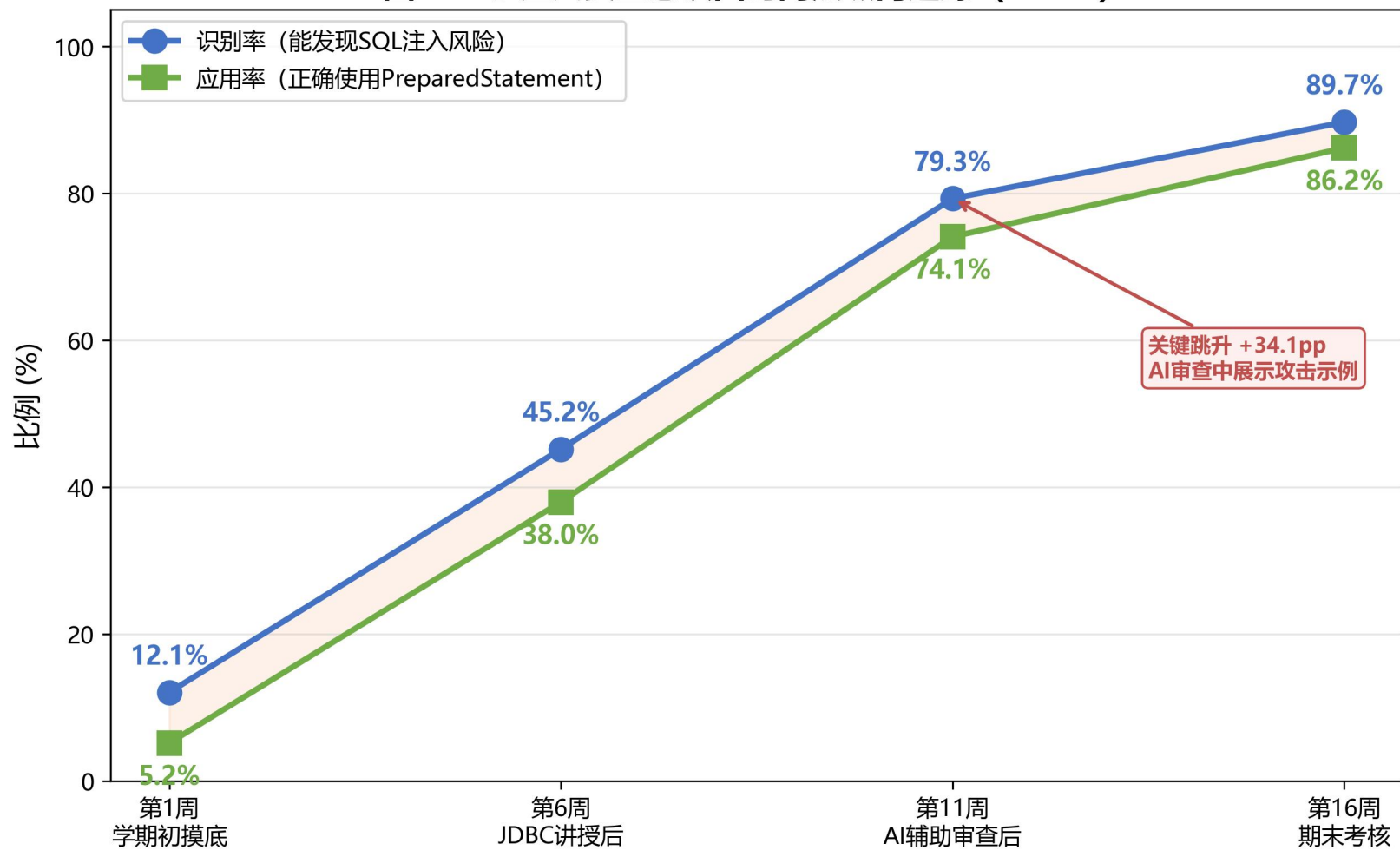


图9 实验组与对照组核心成效指标对比 (N=58 vs N=62)

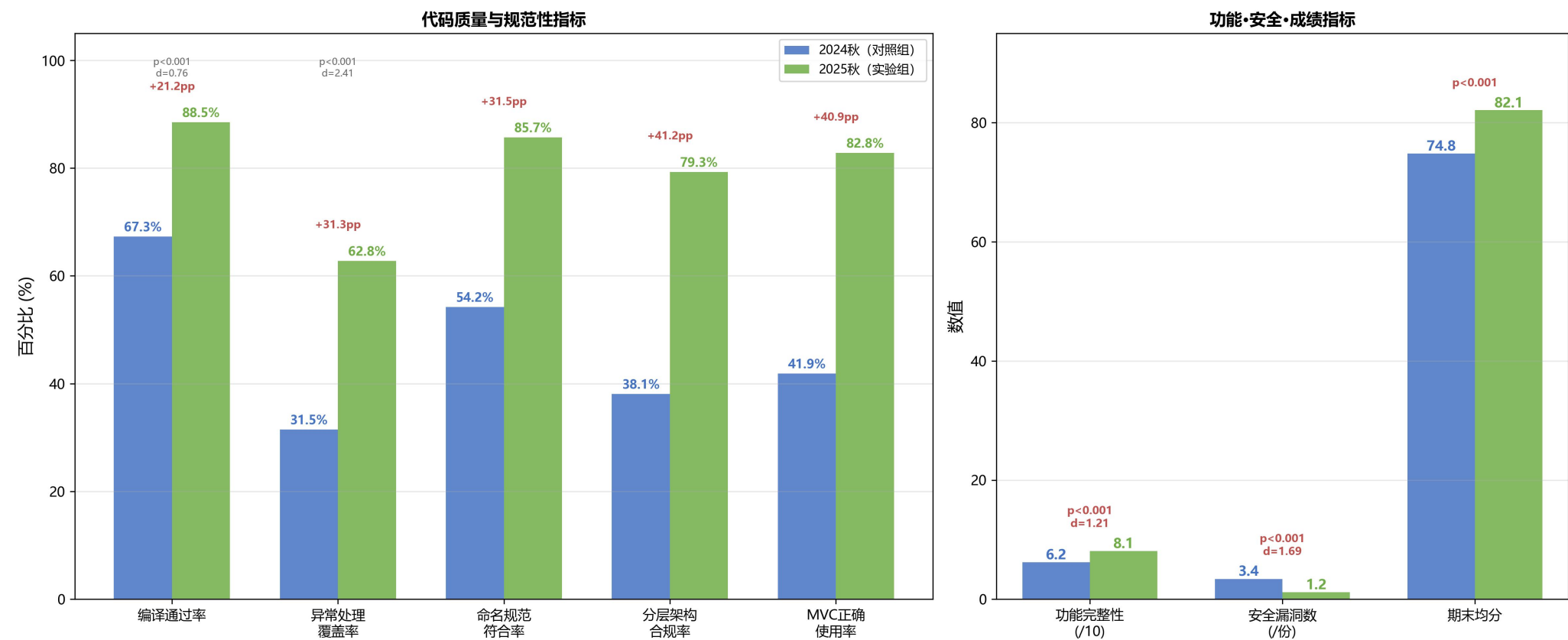


图10 实验组与对照组学业成绩分布对比

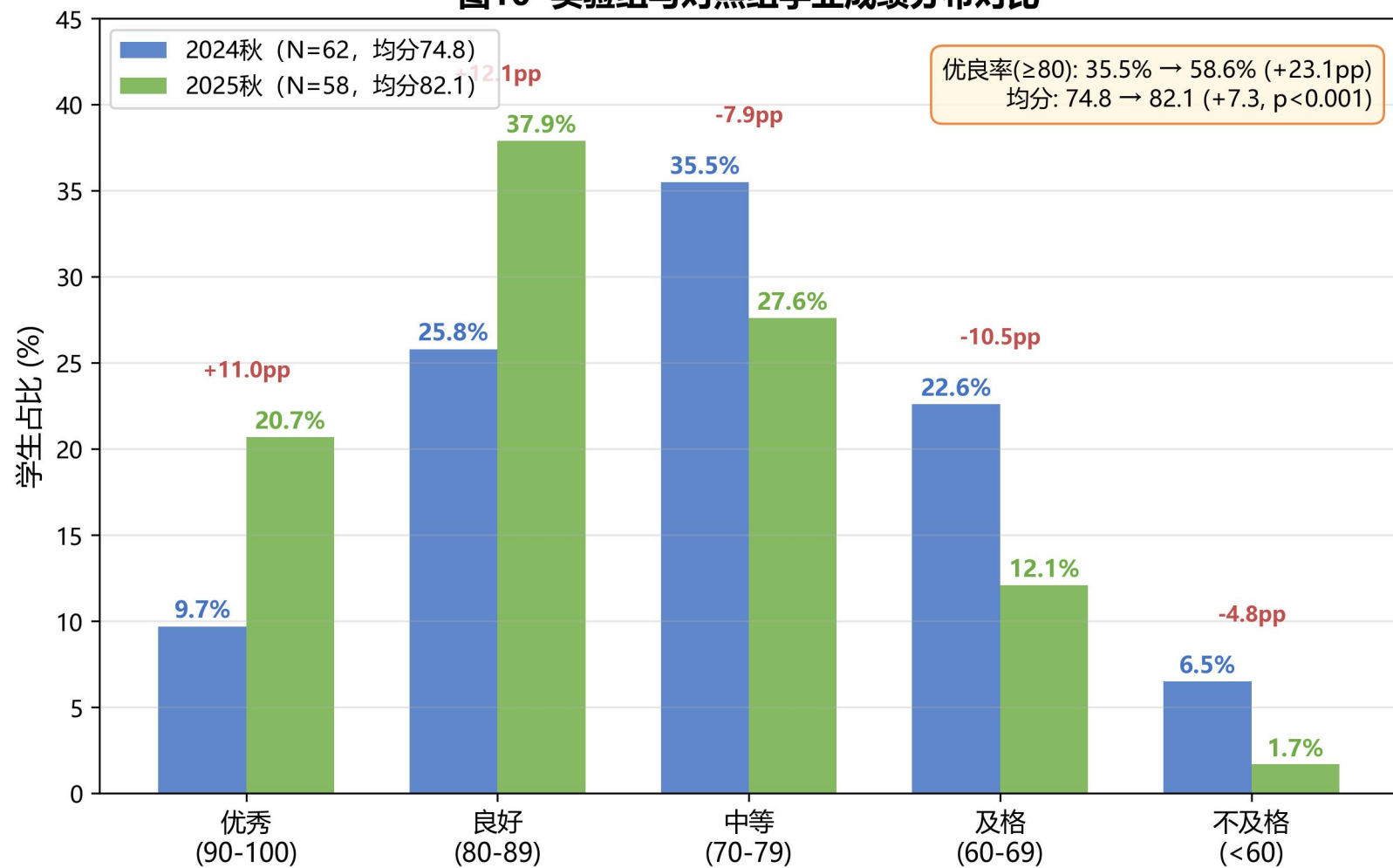


图11 第14次作业五类安全漏洞出现率对比

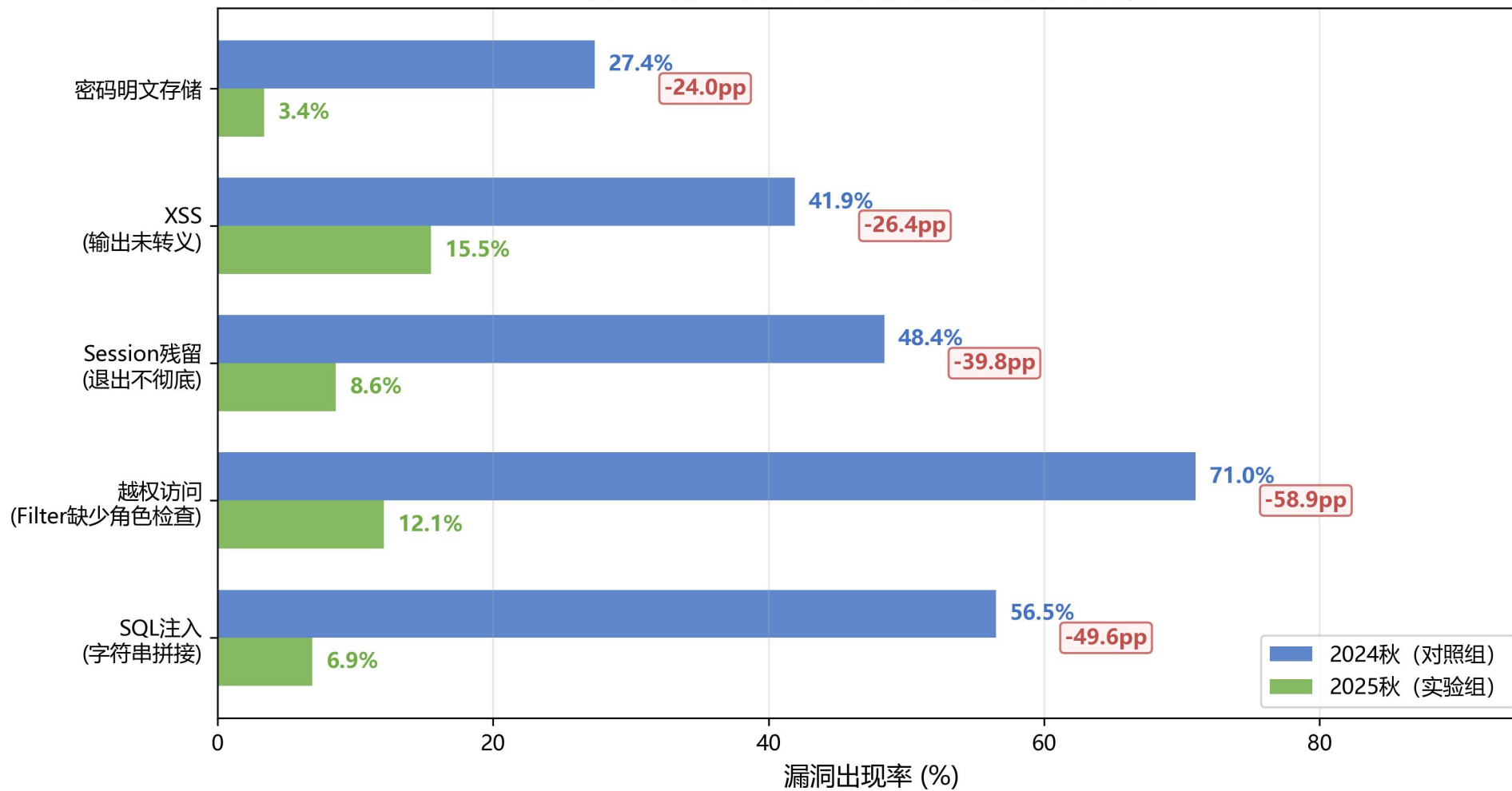
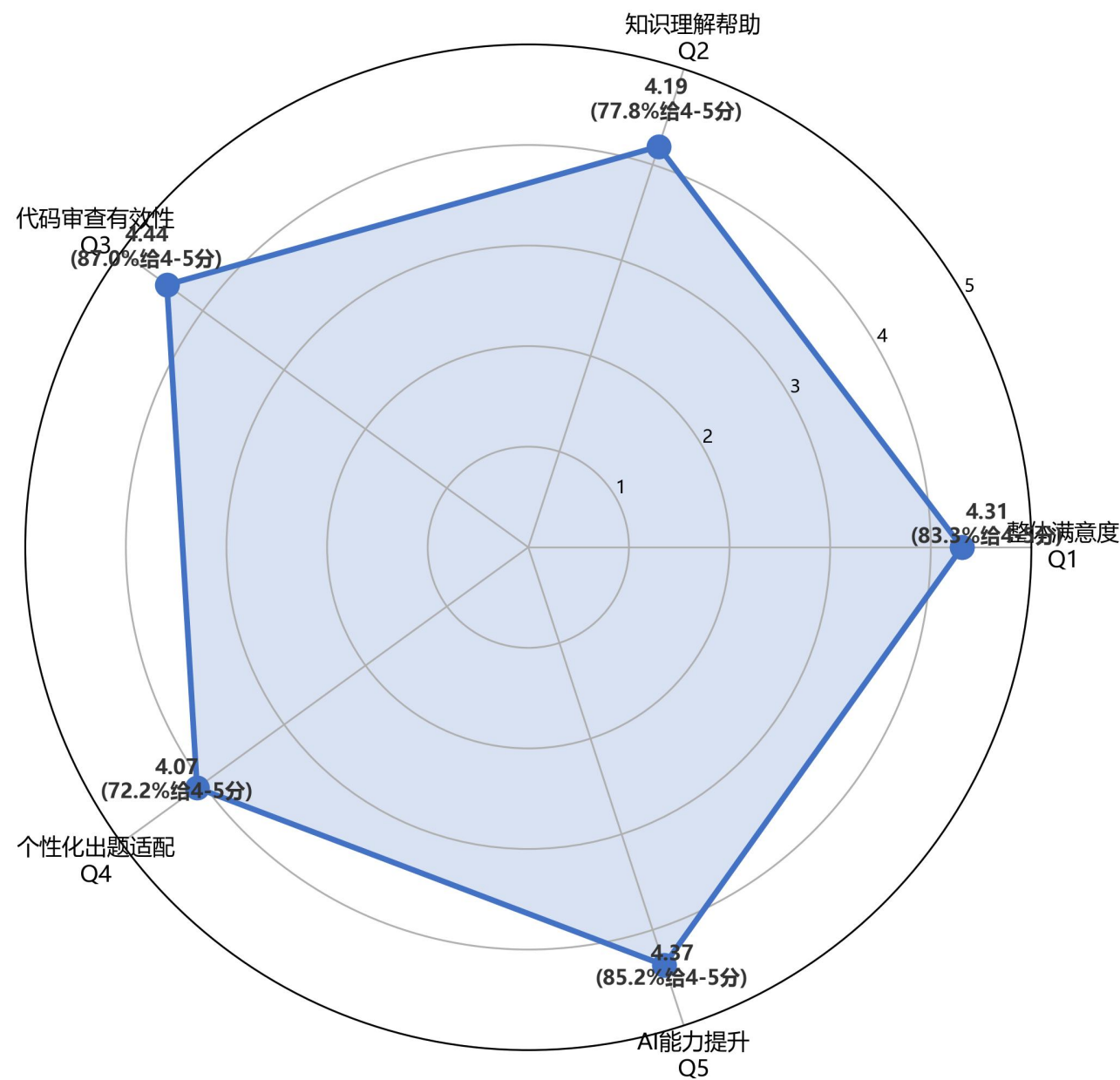


图12 学生问卷满意度多维评价雷达图 (N=54, $\alpha=0.87$)



92.6%的学生愿意在未来课程中继续使用AI辅助学习 (Q15=4.56/5)

三、学生作品与 AI 反馈样例

AI反馈报告样例（标准案例）

学生编号: S2302-15 (已匿名化)

作业批次: 第10次作业——MVC综合实战

作业主题: 实现"课程作业管理系统"的作业发布与查看功能 (个性化任务: 图书借阅管理)

工具: Cursor IDE (内置Claude模型)

使用场景: 提交代码后, 使用AI进行自审查并撰写反馈报告

提交时间: 2025年10月29日 21:47

一、学生提交的代码结构

```
src/main/java/com/tyust/library/
├─ controller/
│   └─ BookServlet.java          ← 问题集中区
├─ dao/
│   └─ BookDao.java             ← 存在SQL注入
├─ model/
│   └─ Book.java
└─ (缺少service包)              ← 架构缺失

src/main/webapp/
├─ WEB-INF/web.xml
├─ book/
│   └─ list.jsp                  ← 含scriptlet代码
│   └─ add.jsp
└─ index.jsp
```

二、AI审查请求

学生输入的提示词

请审查我的Java web MVC项目代码 (图书借阅管理系统), 重点检查以下方面:

- MVC分层是否清晰 (Controller/Service/DAO/Model)
- Servlet中是否存在业务逻辑泄漏 (应在Service层)
- JSP中是否存在Java代码 (应使用EL+JSTL)
- 数据库操作是否正确使用PreparedStatement
- 异常处理是否合理

请按以下格式输出每个问题:

- 文件与行号
- 问题描述
- 严重程度 (高/中/低)
- 对应MVC知识点
- 修改建议

学生附上的代码文件

BookServlet.java (核心问题代码) :

```
@WebServlet("/book")
public class BookServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse
response)
        throws ServletException, IOException {
        request.setCharacterEncoding("UTF-8");
        String action = request.getParameter("action");

        if ("list".equals(action)) {
            // 问题1: Servlet中直接写SQL--业务逻辑泄漏到Controller
            Connection conn = null;
            try {
                conn = DBUtil.getConnection();
                String sql = "SELECT * FROM books ORDER BY id DESC";
                Statement stmt = conn.createStatement();
                ResultSet rs = stmt.executeQuery(sql);
                List<Book> books = new ArrayList<>();
                while (rs.next()) {
                    Book b = new Book();
                    b.setId(rs.getInt("id"));
                    b.setTitle(rs.getString("title"));
                    b.setAuthor(rs.getString("author"));
                    b.setStatus(rs.getString("status"));
                    books.add(b);
                }
                request.setAttribute("books", books);
                request.getRequestDispatcher("/book/list.jsp").forward(request,
response);
            } catch (Exception e) {
                // 问题4: 空catch块--异常被吞掉
            } finally {
                if (conn != null) try { conn.close(); } catch (Exception e) {}
            }

        } else if ("search".equals(action)) {
            // 问题3: SQL字符串拼接--SQL注入风险
            String keyword = request.getParameter("keyword");
            BookDao dao = new BookDao();
            List<Book> results = dao.searchByTitle(keyword);
            request.setAttribute("books", results);
            request.getRequestDispatcher("/book/list.jsp").forward(request,
response);
        }
    }
}
```

BookDao.java (SQL注入问题) :

```
public class BookDao {
```

```

public List<Book> searchByTitle(String title) {
    List<Book> list = new ArrayList<>();
    try {
        Connection conn = DBUtil.getConnection();
        // SQL注入风险: 直接拼接用户输入
        String sql = "SELECT * FROM books WHERE title LIKE '%" + title +
"%'";

        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(sql);
        while (rs.next()) {
            Book b = new Book();
            b.setId(rs.getInt("id"));
            b.setTitle(rs.getString("title"));
            b.setAuthor(rs.getString("author"));
            b.setStatus(rs.getString("status"));
            list.add(b);
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return list;
}
}

```

list.jsp (scriptlet问题) :

```

<%@ page import="java.util.List, com.tyust.library.model.Book" %>
<html>
<body>
<h2>图书列表</h2>
<table border="1">
<%
    // 问题2: JSP中使用scriptlet--应使用EL+JSTL
    List<Book> books = (List<Book>) request.getAttribute("books");
    if (books != null) {
        for (Book b : books) {
%>
            <tr>
                <td><%= b.getId() %></td>
                <td><%= b.getTitle() %></td>
                <td><%= b.getAuthor() %></td>
                <td><%= b.getStatus() %></td>
            </tr>
        <%
            }
        }
    %>
</table>
</body>
</html>

```

三、AI审查输出（完整记录）

我审查了你的图书借阅管理系统代码，发现以下5个问题，按严重程度排序：

问题1 (严重程度：高) -- Service层缺失

文件：项目结构 (无BookService.java)

问题描述：项目中缺少Service层，BookServlet直接操作数据库连接 (第12-29行)，将数据访问逻辑写在了Controller中。这违反了MVC模式中Controller只负责"请求分发"的原则。

对应知识点：MVC模式、三层架构 (Controller → Service → DAO)、单一职责原则

修改建议：

1. 新建 com.tyust.library.service.BookService 类
2. 将BookServlet中的数据库操作迁移到BookService
3. BookService调用BookDao完成数据访问
4. BookServlet只负责：接收参数 → 调用Service → 设置属性 → 转发JSP

问题2 (严重程度：高) -- SQL注入漏洞

文件：BookDao.java 第6行

问题描述：searchByTitle方法使用字符串拼接构造SQL查询：

```
"SELECT * FROM books WHERE title LIKE '%" + title + "%'"
```

如果用户输入 title = "'; DROP TABLE books; --",

拼接后的SQL将变为：

```
SELECT * FROM books WHERE title LIKE '%'; DROP TABLE books; --'
```

这将导致books表被删除。

对应知识点：SQL注入 (SQL Injection)、PreparedStatement参数化查询

修改建议：

```
String sql = "SELECT * FROM books WHERE title LIKE ?";
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setString(1, "%" + title + "%");
ResultSet rs = pstmt.executeQuery();
```

问题3 (严重程度：中) -- JSP中使用scriptlet代码

文件：list.jsp 第6-18行

问题描述：使用<% %>脚本代码和<%= %>表达式输出数据。这种写法将Java逻辑嵌入视图层，违反了MVC的关注点分离原则，且难以维护。

对应知识点：EL表达式、JSTL标签库、JSP最佳实践

修改建议：

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<c:forEach var="book" items="${books}">
    <tr>
        <td>${book.id}</td>
        <td>${book.title}</td>
        <td>${book.author}</td>
        <td>${book.status}</td>
    </tr>
</c:forEach>
```

问题4（严重程度：中）—— 空catch块

文件: BookServlet.java 第27行

问题描述: catch (Exception e) 块为空, 异常信息被完全吞掉。如果数据库查询失败, 用户看到的是空白页面, 开发者看不到任何错误信息, 排查困难。

对应知识点: Java异常处理、Web应用错误处理

修改建议: 至少使用日志记录异常信息:

```
catch (Exception e) {
    System.err.println("查询图书列表失败: " + e.getMessage());
    request.setAttribute("error", "系统繁忙, 请稍后重试");
    request.getRequestDispatcher("/error.jsp").forward(request, response);
}
```

问题5（严重程度：低）—— 建议添加分页功能

文件: BookServlet.java / BookDao.java

问题描述: 当前查询返回所有记录。如果数据量大, 页面加载慢且用户体验差。

对应知识点: 分页查询 (LIMIT/OFFSET)、用户体验

修改建议: 添加分页参数 (page, pageSize), SQL中使用LIMIT。

优先级建议: 先修问题1和2 (架构+安全), 再修问题3和4 (规范+健壮性), 问题5可作为进阶优化。

四、学生自我评估

逐条判断

#	AI建议	我的决策	判断理由	花了多久理解
1	新增Service层	✓ 采纳	第9讲MVC模式课上老师画过三层架构图, Controller确实不应该碰数据库。我重新翻了课堂笔记确认。	约3分钟回忆课堂内容
2	SQL改为PreparedStatement	✓ 采纳	第6讲JDBC课重点讲过SQL注入, AI给的攻击示例 (DROP TABLE) 让我直观感受到了危险性。我之前虽然学过但写代码时忘了。	立即理解, 但惭愧于犯了已知错误
3	JSP改用EL+JSTL	✓ 采纳	第4讲学过EL与JSTL, scriptlet确实不规范。AI给的 <c:forEach> 写法和课堂示例一致。	约2分钟对照课堂代码

#	AI建议	我的决策	判断理由	花了多久理解
4	catch块补充异常处理	✓ 采纳	空catch确实有问题。AI建议的"记录日志+跳转错误页"比我想的"打印堆栈"更合理。	约1分钟
5	添加分页功能	X 拒绝	本次作业要求是MVC基本CRUD，分页不在要求范围内。如果加分页还需要改JSP和DAO，工作量大但不得分。	立即判断拒绝

修改前后diff汇总

项目结构变化：

+ src/main/java/com/tyust/library/service/BookService.java （新增）

BookServlet.java:

- Connection conn = null;

- try { conn = DBUtil.getConnection(); ... } （删除28行数据库操作代码）

+ BookService bookService = new BookService();

+ List<Book> books = bookService.listAll(); （改为调用Service）

BookDao.java:

- String sql = "SELECT * FROM books WHERE title LIKE '%" + title + "%'";

- Statement stmt = conn.createStatement();

+ String sql = "SELECT * FROM books WHERE title LIKE ?";

+ PreparedStatement pstmt = conn.prepareStatement(sql);

+ pstmt.setString(1, "%" + title + "%");

list.jsp:

- <%@ page import="java.util.List, com.tyust.library.model.Book" %>

- <% List<Book> books = (List<Book>) request.getAttribute("books"); ... （删除12行scriptlet）

+ <%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>

+ <c:forEach var="book" items="\${books}"> ... （改为EL+JSTL）

反思（200字）

AI帮我找到了5个问题，其中3个是我**明知道正确做法但写代码时忘记了**（SQL注入、EL表达式、三层架构）。这说明“知道知识点”和“写代码时能正确应用”之间有很大差距。传统的学习方式（听课→做题→考试）很难弥补这个差距，因为你总觉得自己会了，直到AI指出代码中的具体错误行号，才发现自己其实没做到。

AI也给了一个不必要的建议（添加分页）。我拒绝了它，因为本次作业的目标是掌握MVC分层，不是做一个完整产品。如果不加判断地全部采纳，反而偏离了学习重点。

结论：AI审查最大的价值是“把知识点对照到我自己的代码中”——这比看课本例子有效得多。但AI不了解作业要求和课程进度，所以必须自己判断哪些建议该采纳。

五、教师批注

AI使用规范性：★★★★★ (5/5)

提示词结构清晰，按任务卡格式输入，附上了完整代码。

批判性思维：★★★★☆ (4/5)

对5条建议逐条判断，拒绝理由合理。扣1分原因：对问题4的采纳可以更深入——AI建议的 `System.err.println` 在生产中也不够好，应考虑日志框架，但这超出本课程范围，不强求。

知识点对照：★★★★★ (5/5)

每条判断都明确对应了课堂讲次和知识点（"第9讲MVC""第6讲JDBC""第4讲EL"），说明学生在AI辅助下主动回顾了前序知识。

修改质量：★★★★★ (5/5)

Service层补充完整，SQL注入修复正确，JSP改写规范。代码在修改后通过了全部8个功能测试点。

综合评价：这是AI辅助自查的标准范例。学生展示了"AI发现→人工判断→动手修改→回顾知识"的完整闭环。特别值得肯定的是对分页建议的拒绝——能区分"正确但不必要"的建议是AI素养的重要表现。

成绩：本次作业 9.2/10（功能8/8 + 规范1.2/2）

——教师评语，2025年11月1日

AI反馈报告样例（纠偏案例——学生拒绝AI过度建议）

学生编号: S2301-06（已匿名化）

作业批次: 第11次作业——Filter过滤器与请求处理链

作业主题: 实现统一编码过滤器 + 登录验证过滤器（个性化任务: 新闻管理系统）

工具: ChatGPT-4

使用场景: 完成编码后使用AI审查, AI给出了大量"企业级"建议, 学生进行批判性筛选

提交时间: 2025年11月5日 20:23

本案例价值: 展示学生能够拒绝AI的过度设计建议, 体现批判性AI使用能力

一、学生提交的代码

EncodingFilter.java:

```
@WebFilter("/*")
public class EncodingFilter implements Filter {
    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain
chain)
        throws IOException, ServletException {
        req.setCharacterEncoding("UTF-8");
        // 遗漏了response编码设置
        chain.doFilter(req, res);
    }
}
```

LoginCheckFilter.java:

```
@WebFilter("/news/*")
public class LoginCheckFilter implements Filter {
    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain
chain)
        throws IOException, ServletException {
        HttpServletRequest request = (HttpServletRequest) req;
        HttpServletResponse response = (HttpServletResponse) res;
        HttpSession session = request.getSession(false);

        if (session != null && session.getAttribute("user") != null) {
            chain.doFilter(req, res);
        } else {
            response.sendRedirect(request.getContextPath() + "/login.jsp");
        }
    }
}
```

二、AI审查输出（完整记录，含过度建议）

我审查了你的Filter代码，发现以下问题和改进建议：

EncodingFilter审查结果

1. 【问题】Response编码缺失

文件: EncodingFilter.java 第6行

描述: 只设置了request编码，未设置response编码和ContentType。
当Servlet输出中文时可能出现乱码。

严重程度: 中

建议: 添加 `res.setCharacterEncoding("UTF-8");`
以及 `res.setContentType("text/html;charset=UTF-8");`

2. 【问题】编码值硬编码

文件: EncodingFilter.java 第6行

描述: "UTF-8"直接写在代码中，如需更改编码需修改源码并重新编译。

严重程度: 低

建议: 将编码值配置为Filter的init-param:

```
<filter>
  <filter-name>EncodingFilter</filter-name>
  <filter-class>...EncodingFilter</filter-class>
  <init-param>
    <param-name>encoding</param-name>
    <param-value>UTF-8</param-value>
  </init-param>
</filter>
```

然后在init方法中读取:

```
private String encoding;
public void init(FilterConfig config) {
    this.encoding = config.getInitParameter("encoding");
    if (this.encoding == null) this.encoding = "UTF-8";
}
```

3. 【过度建议】引入Spring Security框架

描述: 建议使用Spring Security框架统一管理编码过滤器、认证过滤器、CSRF过滤器等。Spring Security提供了完整的过滤器链管理机制，包括:

- CharacterEncodingFilter (内置编码过滤器)
- UsernamePasswordAuthenticationFilter
- CsrfFilter
- SessionManagementFilter

配置示例:

```
@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {
    @Override protected void configure(HttpSecurity http) { ... }
}
```

4. 【过度建议】使用Redis存储过滤器配置

描述: 建议将过滤器的配置信息（拦截路径、编码值、白名单等）
存储在Redis中，实现运行时动态修改过滤器行为而无需重启服务器。
配置变更后通过消息队列通知各服务器节点刷新配置。

5. 【过度建议】集成Prometheus性能监控
描述：建议在每个Filter的doFilter前后记录执行时间，通过Micrometer暴露metrics，集成Prometheus+Grafana实现过滤器性能实时监控仪表盘。

LoginCheckFilter审查结果

6. 【正确建议】静态资源放行
文件：LoginCheckFilter.java
描述：当前过滤器拦截/news/*下所有请求，包括CSS/JS/图片等静态资源。未登录时静态资源也被拦截，导致登录页样式丢失。
建议：增加白名单判断：

```
String uri = request.getRequestURI();
if (uri.endsWith(".css") || uri.endsWith(".js") || uri.endsWith(".png")) {
    chain.doFilter(req, res);
    return;
}
```

或者将静态资源放在/news/路径之外。

7. 【正确建议】登录页本身也被拦截
文件：LoginCheckFilter.java
描述：如果login.jsp在/news/路径下，访问登录页也会被重定向到登录页，形成无限循环。
建议：在过滤器中排除登录相关路径：

```
if (uri.contains("/login")) {
    chain.doFilter(req, res);
    return;
}
```

三、学生逐条判断与决策

#	AI建议内容	决策	详细理由	对应课程知识
1	Response编码缺失	✓ 采纳	测试时发现确实有中文乱码，添加后解决。这是第11讲课堂上老师演示过的典型问题。	L11 Filter编码处理
2	编码值提取为init-param	✓ 采纳	课堂上老师演示过web.xml配置Filter参数的写法，init-param确实比硬编码更规范。	L11 Filter配置方式
3	引入Spring Security	X 拒绝	本课程使用原生Servlet API，从未涉及Spring框架。引入Spring Security意味着要学习一套全新框架（依赖注入、配置类、安全上下文），这完全偏离了本次作业"理解Filter原理"的目标。	课程范围外

#	AI建议内容	决策	详细理由	对应课程知识
4	使用Redis存储配置	X 拒绝	一个编码过滤器的配置（编码值UTF-8 + 拦截路径/*）只有两个参数，不需要分布式缓存。Redis、消息队列是分布式系统的组件，我的作业是单机Tomcat应用。	过度设计
5	集成Prometheus监控	X 拒绝	这是运维层面的需求，不是开发层面的。一个课堂作业的Filter执行时间在毫秒级，没有监控的必要。Prometheus、Grafana我听都没听过。	完全超出课程范围
6	静态资源放行	✓ 采纳	测试时确实发现登录页CSS丢失，加了白名单判断后恢复正常。	L11 Filter白名单策略
7	登录页排除	✓ 采纳	虽然我的login.jsp不在/news/下所以没触发循环，但AI指出的逻辑隐患确实存在。预防性地加上更安全。	L11 Filter路径配置

决策统计

- **采纳**：4/7（57.1%）——均为与课程知识直接相关的实际问题
- **拒绝**：3/7（42.9%）——均为超出课程范围的"企业级"方案

四、修改后代码

EncodingFilter.java（修改后）：

```
@WebFilter("/*")
public class EncodingFilter implements Filter {
    private String encoding;

    @Override
    public void init(FilterConfig config) {
        this.encoding = config.getInitParameter("encoding");
        if (this.encoding == null) {
            this.encoding = "UTF-8";
        }
    }

    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain)
        throws IOException, ServletException {
        req.setCharacterEncoding(encoding);
        res.setCharacterEncoding(encoding);
        res.setContentType("text/html;charset=" + encoding);
        chain.doFilter(req, res);
    }
}
```

LoginCheckFilter.java (修改后) :

```
@WebFilter("/news/*")
public class LoginCheckFilter implements Filter {
    private static final String[] WHITELIST = {".css", ".js", ".png", ".jpg",
".gif"};

    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain
chain)
        throws IOException, ServletException {
        HttpServletRequest request = (HttpServletRequest) req;
        HttpServletResponse response = (HttpServletResponse) res;

        String uri = request.getRequestURI();

        // 静态资源白名单放行
        for (String suffix : WHITELIST) {
            if (uri.endsWith(suffix)) {
                chain.doFilter(req, res);
                return;
            }
        }

        // 登录页放行
        if (uri.contains("/login")) {
            chain.doFilter(req, res);
            return;
        }

        // 登录状态检查
        HttpSession session = request.getSession(false);
        if (session != null && session.getAttribute("user") != null) {
            chain.doFilter(req, res);
        } else {
            response.sendRedirect(request.getContextPath() + "/login.jsp");
        }
    }
}
```

五、学生反思 (200字)

这次AI给了7条建议，但其中3条（Spring Security、Redis、Prometheus）完全脱离了课程范围。这3条建议本身可能在企业项目中是对的，但放在课堂作业的上下文中就是**过度设计**。

我发现了AI的一个特点：**它不知道我是学生在做课堂作业，它默认你在做生产级项目**。所以它会推荐“最佳实践”而不是“最合适的实践”。这就是为什么老师反复强调“AI是助手不是答案”——如果我不加思考地全部采纳，我的作业会变成一个我完全不理解的“企业级”系统，反而学不到Filter的核心原理。

核心教训：判断AI建议时，要先问自己三个问题：(1)这在课程范围内吗？(2)这解决了实际问题吗？(3)我理解这个建议吗？三个都是“是”才采纳。

六、教师批注

AI使用规范性：★★★★★ (5/5)

按格式提交完整AI交互记录。

批判性思维：★★★★★ (5/5) ——本学期最佳纠偏案例

准确识别出AI的3条过度建议，拒绝理由清晰具体——分别从"课程范围""设计合理性""知识储备"三个角度论证。特别是"AI不知道我是学生做作业"的洞察，说明学生已经建立了成熟的AI协作观。

知识点对照：★★★★☆ (4/5)

采纳的4条建议都正确对应了课程知识点。扣1分：对第7条（登录页排除）的理解可以更深——这不仅是"预防性"措施，而是Filter设计的**必要**考量（否则确实会在某些部署配置下触发循环）。

修改质量：★★★★★ (5/5)

EncodingFilter的init-param改造规范，LoginCheckFilter的白名单实现清晰。白名单使用数组+循环的方式比if-else更可维护——虽然AI没有提到这种实现方式，学生自己选择了更好的写法。

综合评价：这份报告是"AI批判性使用"的教科书案例。学生展示了比"全部采纳"更难的能力——**在正确和过度之间画线**。建议收录为课程教学案例。

成绩：本次作业 9.5/10

——教师评语，2025年11月7日

学生作品证据：代码修改前后对比（样例1）

学生编号：S2301-17（已匿名化）

作业批次：第14次作业——用户认证与权限控制

任务：审查并修复RoleBasedAuthFilter中的权限缺陷

AI工具：Cursor（Claude模型）

修改前代码（学生初始提交）

```
@webFilter("/admin/*")
public class RoleBasedAuthFilter implements Filter {
    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain
chain)
        throws IOException, ServletException {
        HttpServletRequest request = (HttpServletRequest) req;
        HttpServletResponse response = (HttpServletResponse) res;
        HttpSession session = request.getSession(false);

        // 只检查了是否登录，没有检查角色
        boolean loggedIn = session != null
            && session.getAttribute("user") != null;

        if (loggedIn) {
            chain.doFilter(req, res); // 所有登录用户都能进入
        } else {
            response.sendRedirect(request.getContextPath() + "/login.jsp");
        }
    }
}
```

存在的问题

- 只判断 user 是否存在（认证），未判断角色是否为ADMIN（授权）
- 普通用户（USER角色）登录后也能访问 /admin/* 路径
- 混淆了Authentication和Authorization的概念

AI审查输出（学生使用的提示词与AI回复摘要）

学生输入的提示词

请审查这个Java web角色权限过滤器，重点检查：

- 是否只判断了“已登录”，而没有判断角色
- 是否正确识别ADMIN用户
- 普通用户是否仍可能访问/admin/*路径

请按以下格式输出：

- 问题位置
- 风险说明

- 对应知识点
- 修改建议
- 建议验证方式

AI回复（摘要）

问题位置：doFilter方法第11-13行

风险说明：当前只检查session中是否存在user属性，任何已登录用户（包括USER角色）都可以通过过滤器访问/admin/*路径，存在越权访问风险。

对应知识点：Authorization（授权）、RBAC（基于角色的访问控制）

修改建议：获取User对象后检查其角色属性，仅当角色为ADMIN时放行。

建议验证方式：分别用ADMIN和USER角色登录，访问/admin/panel，验证USER被拒绝、ADMIN被放行。

学生判断与决策

"AI指出了过滤器只做了认证没做授权，这个判断是正确的。我决定采纳这个建议，因为课堂上老师讲过Authentication和Authorization的区别，这里确实只做了前者。"

——S2301-17课堂记录

修改后代码（学生最终提交）

```
@WebFilter("/admin/*")
public class RoleBasedAuthFilter implements Filter {
    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain
chain)
        throws IOException, ServletException {
        HttpServletRequest request = (HttpServletRequest) req;
        HttpServletResponse response = (HttpServletResponse) res;
        HttpSession session = request.getSession(false);

        if (session == null) {
            response.sendRedirect(request.getContextPath() + "/login.jsp");
            return;
        }

        User user = (User) session.getAttribute("user");
        if (user != null && "ADMIN".equals(user.getRole())) {
            chain.doFilter(req, res);
        } else if (user != null) {
            response.sendError(HttpServletResponse.SC_FORBIDDEN);
        } else {
            response.sendRedirect(request.getContextPath() + "/login.jsp");
        }
    }
}
```

改进点

- 1. 从Session中获取User对象，检查 `user.getRole()` 是否为ADMIN
- 2. 区分三种情况：未登录→重定向、已登录非ADMIN→403、ADMIN→放行
- 3. 正确实现了Authorization逻辑，符合RBAC模型

测试验证结果

测试场景	操作	预期结果	实际结果	通过
未登录访问	直接访问/admin/panel	重定向登录页	重定向登录页	✓
USER访问	tom登录后访问/admin/panel	403拒绝	403拒绝	✓
ADMIN访问	admin登录后访问/admin/panel	正常进入	正常进入	✓

学生作品证据：代码修改前后对比（样例2）

学生编号：S2302-08（已匿名化）

作业批次：第14次作业——用户认证与权限控制

任务：审查并修复LogoutServlet中的Session管理缺陷

AI工具：ChatGPT-4

修改前代码（学生初始提交）

```
@webServlet("/logout")
public class LogoutServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws IOException {
        HttpSession session = req.getSession(false);
        if (session != null) {
            session.removeAttribute("user");
        }
        resp.sendRedirect(req.getContextPath() + "/login.jsp");
    }
}
```

存在的问题

- 只移除了user属性，Session中其他属性（如isLoggedIn、role）仍然残留
- Session本身未销毁，JSESSIONID仍然有效
- 退出后如果其他代码依赖isLoggedIn判断，用户仍可能被误认为已登录

AI审查输出（摘要）

AI回复关键内容

问题位置：doGet方法第5行 session.removeAttribute("user")
风险说明：removeAttribute仅删除单个属性，Session对象本身仍然存活。如果Session中存储了多个属性（如isLoggedIn、role），这些属性不会被清除，可能导致"幽灵登录"状态——系统认为用户仍处于登录状态。
对应知识点：Session生命周期管理、Logout安全
修改建议：使用session.invalidate()替代removeAttribute，彻底销毁整个Session。
建议验证方式：退出登录后，通过浏览器开发者工具检查JSESSIONID是否仍然有效，尝试直接访问受保护资源。

学生判断与决策

"AI提到了'幽灵登录'这个问题，我一开始不太理解。后来在浏览器开发者工具中看了退出前后的Cookie变化，发现退出后JSESSIONID确实没变，说明Session还活着。所以我采纳了invalidate的建议。AI还提到可以在登录时重新生成Session ID防止固定会话攻击，但我觉得这超出了本次作业范围，先不做。"

——S2302-08课堂记录

修改后代码（学生最终提交）

```
@webServlet("/logout")
public class LogoutServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws IOException {
        HttpSession session = req.getSession(false);
        if (session != null) {
            session.invalidate();
        }
        resp.sendRedirect(req.getContextPath() + "/login.jsp");
    }
}
```

改进点

- 1. 使用 invalidate() 彻底销毁Session，所有属性一并清除
- 2. JSESSIONID 失效，后续请求会创建新Session
- 3. 消除了"幽灵登录"风险

测试验证结果

测试场景	操作	预期结果	实际结果	通过
退出后Session销毁	退出后检查Cookie	JSESSIONID变化	JSESSIONID已失效，新请求获得新ID	✓
退出后访问后台	退出后直接访问/admin/panel	重定向登录页	重定向登录页	✓
退出后首页状态	退出后访问index.jsp	显示"未登录"	显示"未登录"	✓

学生作品证据：代码修改前后对比（样例3）

学生编号: S2301-24 (已匿名化)

作业批次: 第6次作业——JDBC数据库连接

任务: 实现用户查询功能, 并确保SQL安全

AI工具: Cursor (Claude模型)

修改前代码（学生初始提交）

```
public class UserDao {  
    public User findByUsername(String username) {  
        Connection conn = null;  
        Statement stmt = null;  
        ResultSet rs = null;  
        try {  
            conn = DBUtil.getConnection();  
            // 直接拼接SQL——存在SQL注入风险  
            String sql = "SELECT * FROM users WHERE username = '" + username +  
            ""';  
            stmt = conn.createStatement();  
            rs = stmt.executeQuery(sql);  
            if (rs.next()) {  
                User user = new User();  
                user.setUsername(rs.getString("username"));  
                user.setPasswordHash(rs.getString("password_hash"));  
                user.setRole(rs.getString("role"));  
                return user;  
            }  
        } catch (SQLException e) {  
            e.printStackTrace(); // 直接打印堆栈——生产环境不推荐  
        } finally {  
            // 未关闭资源  
        }  
        return null;  
    }  
}
```

存在的问题

- SQL字符串拼接, 存在SQL注入漏洞 (输入 ' OR '1'='1 可绕过查询)
- 使用Statement而非PreparedStatement
- 异常处理粗糙 (e.printStackTrace())
- finally块未关闭Connection/Statement/ResultSet, 存在资源泄漏

AI审查输出（摘要）

问题1 (严重): SQL注入漏洞

位置: 第7行SQL拼接

风险：攻击者可通过构造username参数执行任意SQL

知识点：SQL Injection、PreparedStatement

建议：使用PreparedStatement + 参数化查询

问题2（中等）：资源泄漏

位置：finally块为空

风险：数据库连接未释放，长期运行后连接池耗尽

知识点：JDBC资源管理、try-with-resources

建议：使用try-with-resources自动关闭

问题3（低级）：异常处理不当

位置：catch块第14行

风险：堆栈信息可能暴露数据库结构

知识点：异常处理最佳实践

建议：使用日志框架记录，不直接打印堆栈

学生判断与决策

"AI指出了三个问题，我全部采纳。SQL注入是课堂上老师重点讲过的，我竟然自己写的时候还是犯了这个错误。资源泄漏的问题我之前没意识到，AI的提醒让我意识到finally块不能留空。异常处理的问题我暂时用简单的日志替代，以后学了日志框架再改进。"

——S2301-24作业反思

修改后代码（学生最终提交）

```
public class UserDao {
    public User findByUsername(String username) {
        String sql = "SELECT * FROM users WHERE username = ?";
        try (Connection conn = DBUtil.getConnection();
            PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setString(1, username);
            try (ResultSet rs = pstmt.executeQuery()) {
                if (rs.next()) {
                    User user = new User();
                    user.setUsername(rs.getString("username"));
                    user.setPasswordHash(rs.getString("password_hash"));
                    user.setRole(rs.getString("role"));
                    return user;
                }
            }
        } catch (SQLException e) {
            System.err.println("查询用户失败: " + e.getMessage());
        }
        return null;
    }
}
```

改进点

1. 使用PreparedStatement + 参数化查询，消除SQL注入风险
2. 使用try-with-resources自动关闭所有JDBC资源
3. 异常处理输出简洁错误信息，不暴露堆栈细节

实验报告样例：基础层

学生编号：S2302-22（已匿名化）

实验名称：用户认证与权限控制——基础任务

难度层级：基础（Basic）

任务：将密码哈希从SHA-256升级为BCrypt

一、实验目标

将课堂示例工程中的 `PasswordUtil` 从SHA-256升级为BCrypt算法，理解两种算法在安全性上的差异。

二、实验环境

- JDK 8、Maven 3.8、Tomcat 9
- 新增依赖：`jbcrypt 0.4`

三、实验步骤

3.1 添加BCrypt依赖

```
<dependency>
  <groupId>org.mindrot</groupId>
  <artifactId>jbcrypt</artifactId>
  <version>0.4</version>
</dependency>
```

3.2 修改PasswordUtil

```
public final class PasswordUtil {
    public static String hashPassword(String password) {
        return BCrypt.hashpw(password, BCrypt.gensalt(12));
    }

    public static boolean matches(String rawPassword, String hashedPassword) {
        return BCrypt.checkpw(rawPassword, hashedPassword);
    }
}
```

3.3 更新UserDao中的预置用户密码

```
static {
    Map<String, User> temp = new HashMap<>();
    temp.put("admin", new User("admin",
        PasswordUtil.hashPassword("admin123"), "ADMIN", "课程管理员"));
    temp.put("tom", new User("tom",
        PasswordUtil.hashPassword("tom123"), "USER", "普通学生 Tom"));
    USERS = Collections.unmodifiableMap(temp);
}
```

四、测试结果

测试场景	操作	预期结果	实际结果	通过
正确密码登录	admin/admin123登录	登录成功	登录成功	✓
错误密码登录	admin/wrong登录	提示错误	提示"用户名或密码错误"	✓
哈希不可逆	查看存储的密码哈希	不可还原为明文	BCrypt哈希值每次不同	✓

五、SHA-256 vs BCrypt对比分析

维度	SHA-256	BCrypt
算法类型	消息摘要（快速）	自适应哈希（慢速）
盐值	需要手动加盐	自动内置盐值
计算速度	极快（GPU可加速）	故意慢（防暴力破解）
彩虹表攻击	无盐时易受攻击	每次哈希不同，无法预计算
适用场景	数据完整性校验	密码存储

六、实验总结

通过本次实验，我理解了为什么课堂上老师说"SHA-256用于演示原理，生产环境应使用BCrypt"。BCrypt的关键优势是**故意设计得慢**——这对正常登录影响微乎其微（约100ms），但对暴力破解攻击来说，每次尝试都要付出100ms的代价，使得暴力破解在实际中不可行。

七、AI使用说明

本次实验未使用AI辅助。BCrypt的使用方法参考了jBCrypt官方文档和课堂笔记。

实验报告样例：进阶层

学生编号: S2301-11 (已匿名化)

实验名称: 用户认证与权限控制——进阶任务

难度层级: 进阶 (Advanced)

任务: 增加MANAGER角色, 实现三级权限 (USER/MANAGER/ADMIN)

一、实验目标

在现有双角色 (USER/ADMIN) 的基础上, 新增MANAGER角色, 实现三级权限控制:

- USER: 仅访问首页和个人页面
- MANAGER: 可访问 `/report/*` 报表页面
- ADMIN: 可访问 `/admin/*` 后台管理

二、设计思路

2.1 角色与路径映射

角色	可访问路径	不可访问路径
USER	<code>/</code> , <code>/index.jsp</code> , <code>/user/*</code>	<code>/report/*</code> , <code>/admin/*</code>
MANAGER	<code>/</code> , <code>/index.jsp</code> , <code>/user/*</code> , <code>/report/*</code>	<code>/admin/*</code>
ADMIN	所有路径	无

2.2 新增文件

- `ReportAuthFilter.java` — 拦截 `/report/*`, 只允许MANAGER和ADMIN
- `report/dashboard.jsp` — 报表页面
- 修改 `UserDao` — 新增MANAGER预置用户
- 修改 `index.jsp` — 按角色显示三种菜单

三、核心代码

3.1 ReportAuthFilter

```
@WebFilter("/report/*")
public class ReportAuthFilter implements Filter {
    private static final Set<String> ALLOWED_ROLES =
        new HashSet<>(Arrays.asList("MANAGER", "ADMIN"));

    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain
chain)
        throws IOException, ServletException {
        HttpServletRequest request = (HttpServletRequest) req;
        HttpServletResponse response = (HttpServletResponse) res;
```

```
HttpSession session = request.getSession(false);

if (session == null) {
    response.sendRedirect(request.getContextPath() + "/login.jsp");
    return;
}

User user = (User) session.getAttribute("user");
if (user != null && ALLOWED_ROLES.contains(user.getRole())) {
    chain.doFilter(req, res);
} else if (user != null) {
    response.sendError(HttpServletResponse.SC_FORBIDDEN);
} else {
    response.sendRedirect(request.getContextPath() + "/login.jsp");
}
}
```

3.2 index.jsp菜单（关键片段）

```
<c:if test="${sessionScope.user.role == 'MANAGER'
|| sessionScope.user.role == 'ADMIN'}">
    <a href="${pageContext.request.contextPath}/report/dashboard.jsp">查看报表</a>
</c:if>
<c:if test="${sessionScope.user.role == 'ADMIN'}">
    <a href="${pageContext.request.contextPath}/admin/panel.jsp">管理后台</a>
</c:if>
```

3.3 新增预置用户

```
temp.put("manager", new User("manager",
    PasswordUtil.sha256("manager123"), "MANAGER", "部门经理 李华"));
```

四、测试结果

测试场景	用户	访问路径	预期	实际	通过
USER访问报表	tom/USER	/report/dashboard	403	403	✓
USER访问后台	tom/USER	/admin/panel	403	403	✓
MANAGER访问报表	manager/MANAGER	/report/dashboard	正常	正常	✓
MANAGER访问后台	manager/MANAGER	/admin/panel	403	403	✓
ADMIN访问报表	admin/ADMIN	/report/dashboard	正常	正常	✓
ADMIN访问后台	admin/ADMIN	/admin/panel	正常	正常	✓

测试场景	用户	访问路径	预期	实际	通过
菜单一致性 (USER)	tom	首页菜单	无报表、无后台	正确	✓
菜单一致性 (MANAGER)	manager	首页菜单	有报表、无后台	正确	✓
菜单一致性 (ADMIN)	admin	首页菜单	有报表、有后台	正确	✓

五、AI使用说明

在设计 `ReportAuthFilter` 时，我使用Cursor询问了"如何让Filter支持多个角色"，AI建议使用 `Set<String>` 存储允许的角色列表，比写多个if-else更优雅。我采纳了这个建议，因为它确实提高了代码的可扩展性——未来增加新角色只需修改Set即可。

六、实验总结

三级权限的关键挑战是**Filter的复用性**。如果每个路径写一个独立的Filter，代码会大量重复。通过使用 `Set<String>` 存储允许角色，可以让Filter更通用。未来如果角色更多，可以考虑将"路径→角色"映射存储在配置文件或数据库中，实现真正的动态RBAC。

实验报告样例：挑战层

学生编号: S2302-03 (已匿名化)

实验名称: 用户认证与权限控制——挑战任务

难度层级: 挑战 (Challenge)

任务: 增强Session安全 (防固定会话攻击、设置超时、实现"记住我")

一、实验目标

- 防御固定会话攻击 (Session Fixation)
- 实现Session超时配置
- 实现"记住我" (Remember Me) 功能

二、安全威胁分析

固定会话攻击原理

攻击者 → 获取一个合法的JSESSIONID (如访问登录页)
攻击者 → 将该JSESSIONID植入受害者浏览器 (如通过URL)
受害者 → 使用该JSESSIONID登录
攻击者 → 使用同一JSESSIONID访问系统 → 以受害者身份操作

防御: 登录成功后, 销毁旧Session, 创建新Session, 迁移必要属性。

三、核心实现

3.1 防固定会话攻击 (LoginServlet改造)

```
@Override
protected void doPost(HttpServletRequest req, HttpServletResponse resp)
    throws ServletException, IOException {
    req.setCharacterEncoding("UTF-8");
    String username = req.getParameter("username");
    String password = req.getParameter("password");

    User user = userService.login(username, password);
    if (user != null) {
        // 防固定会话攻击: 销毁旧Session, 创建新Session
        HttpSession oldSession = req.getSession(false);
        if (oldSession != null) {
            oldSession.invalidate();
        }
        HttpSession newSession = req.getSession(true);
        newSession.setAttribute("user", user);
        newSession.setMaxInactiveInterval(30 * 60); // 30分钟超时

        // 处理"记住我"
        String rememberMe = req.getParameter("rememberMe");
        if ("on".equals(rememberMe)) {
            String token = generateRememberToken(user.getUsername());
```

```

        saveRememberToken(user.getUsername(), token);
        Cookie cookie = new Cookie("remember_token", token);
        cookie.setMaxAge(7 * 24 * 60 * 60); // 7天
        cookie.setHttpOnly(true);
        cookie.setPath(req.getContextPath());
        resp.addCookie(cookie);
    }

    resp.sendRedirect(req.getContextPath() + "/index.jsp");
    return;
}

req.setAttribute("error", "用户名或密码错误");
req.getRequestDispatcher("/login.jsp").forward(req, resp);
}

```

3.2 Remember Token生成

```

private String generateRememberToken(String username) {
    String raw = username + System.currentTimeMillis() + UUID.randomUUID();
    return PasswordUtil.sha256(raw);
}

// 简化实现：使用内存Map存储（生产环境应存数据库）
private static final Map<String, String> REMEMBER_TOKENS =
    new ConcurrentHashMap<>();

private void saveRememberToken(String username, String token) {
    REMEMBER_TOKENS.put(token, username);
}

public static String getUsernameByToken(String token) {
    return REMEMBER_TOKENS.get(token);
}

```

3.3 Session超时配置 (web.xml)

```

<session-config>
    <session-timeout>30</session-timeout> <!-- 30分钟 -->
    <cookie-config>
        <http-only>true</http-only>
    </cookie-config>
</session-config>

```

3.4 自动登录过滤器 (RememberMeFilter)

```

@WebFilter("/*")
public class RememberMeFilter implements Filter {
    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain
chain)
        throws IOException, ServletException {
        HttpServletRequest request = (HttpServletRequest) req;

```

```
HttpSession session = request.getSession(false);

// 如果已登录, 直接放行
if (session != null && session.getAttribute("user") != null) {
    chain.doFilter(req, res);
    return;
}

// 检查remember_token Cookie
Cookie[] cookies = request.getCookies();
if (cookies != null) {
    for (Cookie cookie : cookies) {
        if ("remember_token".equals(cookie.getName())) {
            String username =
LoginServlet.getUsernameByToken(cookie.getValue());
            if (username != null) {
                User user = new UserDao().findByUsername(username);
                if (user != null) {
                    HttpSession newSession = request.getSession(true);
                    newSession.setAttribute("user", user);
                }
            }
            break;
        }
    }
}
chain.doFilter(req, res);
}
```

四、测试结果

测试场景	操作	预期	实际	通过
固定会话防御	登录前后对比JSESSIONID	ID不同	登录后JSESSIONID已更换	✓
Session超时	登录后等待31分钟	Session过期需重新登录	31分钟后访问提示登录	✓
记住我-勾选	勾选"记住我"登录	关闭浏览器后重新打开仍登录	Cookie有效, 自动登录	✓
记住我-未勾选	不勾选"记住我"登录	关闭浏览器后需重新登录	无remember Cookie	✓
记住我-退出	登录后点退出	remember Cookie被清除	Cookie已清除	✓
HttpOnly	浏览器控制台尝试读取Cookie	document.cookie无法读取	HttpOnly生效	✓

五、AI使用说明

1. 使用Cursor询问"Java Web如何防御Session Fixation", AI建议"登录后invalidate旧Session再创建新Session"。我采纳了这个建议。
2. 使用Claude询问"Remember Me功能的安全设计", AI给出了完整的Token方案。我采纳了核心思路, 但简化了Token存储(用内存Map代替数据库), 因为本次实验重点是理解原理。
3. AI额外建议"使用JWT替代Cookie-Token方案"。我拒绝了, 因为JWT涉及更多概念(签名、过期、刷新), 超出本次实验范围。

六、实验总结

Session安全看似简单, 实际涉及多个攻击面。最重要的防御原则是:

1. **登录后更换Session ID** — 防固定会话攻击
2. **合理设置超时** — 防遗忘退出导致的会话劫持
3. **Remember Me Token必须随机且不可预测** — 防Token伪造
4. **Cookie设置HttpOnly** — 防XSS窃取Cookie

这些都是"看不见的安全"——用户感知不到, 但缺少任何一项都可能导致严重的安全事故。